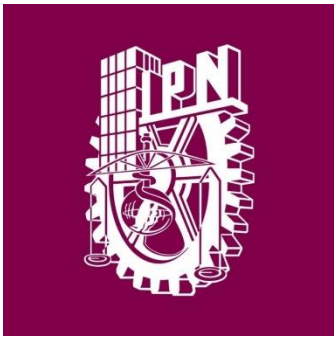




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 5

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

La gestión de procesos en un sistema operativo permite ejecutar múltiples aplicaciones simultáneamente, optimizando el uso de recursos como CPU, memoria y dispositivos de entrada/salida. Windows y Linux ofrecen herramientas específicas para esta tarea, cada uno con enfoques distintos pero igualmente eficaces para mantener el sistema estable y eficiente.

Concepto de Proceso

Un proceso es una instancia de un programa en ejecución. Incluye el código, variables y estado de ejecución del programa. El sistema operativo controla el ciclo de vida de los procesos (creación, ejecución, espera, y finalización), distribuyendo los recursos y evitando que uno solo acapare el sistema.

Administración de Procesos en Linux

- **Control detallado:** Linux permite a los usuarios gestionar cada aspecto de los procesos, como el uso de CPU, memoria y el usuario que los ejecuta.
- **Priorización de Procesos:** Los administradores pueden asignar diferentes niveles de prioridad para garantizar que los procesos críticos reciban los recursos necesarios.
- **Demonios y Servicios:** Linux utiliza procesos en segundo plano llamados demonios para manejar funciones esenciales (como redes o bases de datos). Estos pueden configurarse para arrancar automáticamente con el sistema.
- **Monitoreo en Tiempo Real:** Linux proporciona herramientas avanzadas para observar el rendimiento y consumo de recursos de cada proceso en tiempo real, facilitando la detección de problemas.

Administración de Procesos en Windows

- **Interfaz Gráfica (Administrador de Tareas):** Windows facilita la administración mediante una interfaz visual donde se pueden ver los procesos activos y su consumo de recursos, y finalizar aquellos que consuman demasiados recursos.
- **PowerShell para Tareas Avanzadas:** PowerShell permite a los administradores ejecutar scripts para automatizar tareas complejas en varios dispositivos, ideal para entornos empresariales.
- **Servicios en Segundo Plano:** Windows también usa servicios en segundo plano que cumplen funciones críticas. Se gestionan en la consola de servicios, donde se pueden iniciar o detener según se necesite.

- **Ajuste de Prioridades:** Los usuarios pueden ajustar las prioridades de los procesos para asegurar que los programas importantes tengan los recursos necesarios, maximizando el rendimiento.

Comparación de Enfoques

- **Flexibilidad en Linux:** La estructura modular de Linux permite un control detallado de cada proceso, ideal para servidores y entornos donde se necesita alto grado de personalización.
- **Simplicidad en Windows:** Windows apuesta por la simplicidad en la gestión de procesos con su interfaz gráfica, pero también ofrece herramientas avanzadas (PowerShell) para tareas complejas.

Sección Linux:

1.- Introduzca los siguientes comandos a través de la consola del sistema operativo Linux:

`ps`

`ps -fea`

`ps tree 0`

¿Qué información le proporcionan los comandos anteriores?

1. `ps`

El comando `ps` sin argumentos ofrece información sobre los procesos de la sesión de terminal actual del usuario, proporcionando una visión limitada, pero útil, del entorno inmediato. En su salida, verás columnas típicas como:

- **PID (Process ID):** Identificador único de cada proceso. Es fundamental para gestionar el proceso (ej., terminarlo).
- **TTY (Terminal Type):** La terminal a la que está asociado el proceso. Esto indica desde qué consola o terminal fue lanzado el proceso.
- **TIME:** Tiempo acumulado que el proceso ha consumido en la CPU, medido en minutos y segundos.
- **CMD (Command):** El comando ejecutado para iniciar el proceso.

```
said@said-VirtualBox:~$ ps
  PID TTY          TIME CMD
 2878 pts/0    00:00:00 bash
 2909 pts/0    00:00:00 ps
```

2. ps -fea

Cuando se usa `ps -fea`, se obtiene una lista mucho más detallada y completa de todos los procesos en ejecución en el sistema, no solo los de la sesión actual. A continuación se detallan las columnas que aparecen:

- **UID (User ID):** El nombre del usuario que lanzó el proceso. Esto permite identificar el propietario de cada proceso, útil en sistemas multiusuario.
- **PID (Process ID):** El identificador del proceso, único en el sistema.
- **PPID (Parent Process ID):** El ID del proceso padre que lanzó el proceso actual. Esta relación permite rastrear la jerarquía de procesos y saber qué proceso originó cada tarea.
- **C (CPU Usage):** Prioridad de uso de la CPU asignada al proceso, que puede influir en el rendimiento y la cantidad de CPU que consume.
- **STIME (Start Time):** Hora en la que se inició el proceso.
- **TTY (Terminal Type):** La terminal asociada al proceso. Si no tiene terminal, indicará ?.
- **TIME:** Tiempo acumulado de CPU que el proceso ha utilizado.
- **CMD (Command):** El nombre del comando o programa en ejecución, junto con sus argumentos.

```
said@said-VirtualBox:~$ ps -fea
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	3	11:21	?	00:00:02	/sbin/init splash
root	2	0	0	11:21	?	00:00:00	[kthreadd]
root	3	2	0	11:21	?	00:00:00	[pool_workqueue_release]
root	4	2	0	11:21	?	00:00:00	[kworker/R-rcu_g]
root	5	2	0	11:21	?	00:00:00	[kworker/R-rcu_p]
root	6	2	0	11:21	?	00:00:00	[kworker/R-slub_]
root	7	2	0	11:21	?	00:00:00	[kworker/R-netns]
root	8	2	0	11:21	?	00:00:00	[kworker/0:0-cgroup_destroy]
root	9	2	0	11:21	?	00:00:00	[kworker/0:1-events]
root	10	2	0	11:21	?	00:00:00	[kworker/0:0H-events_highpri]
root	11	2	0	11:21	?	00:00:00	[kworker/u8:0-events_unbound]

3. pstree 0

El comando `pstree` presenta los procesos en forma de árbol, mostrando visualmente las relaciones jerárquicas. El argumento `0` especifica que el árbol debe empezar desde el proceso raíz (PID 0), que es el "padre" de todos los procesos del sistema, aunque el proceso que comúnmente ocupa la raíz en Linux es `systemd` (PID 1), que a su vez es el primer proceso que arranca en el sistema.

- **Visualización jerárquica:** La estructura de árbol que muestra pstree permite ver de qué procesos dependen otros. Por ejemplo, es común ver systemd o init en la raíz, seguido de servicios del sistema y demonios, luego shells de usuario, y más abajo, procesos individuales.
- **Relaciones padre-hijo:** Puedes ver qué proceso originó a otro. Por ejemplo, una sesión de ssh iniciada por un usuario podría tener varios comandos y scripts ejecutándose en su entorno; pstree mostrará esos procesos como "hijos" de ssh.

2. A través de la ayuda en línea que proporciona Linux, investigue para que se utilizan los comandos ps, y pstree, mencione las opciones que se pueden utilizar con dichos comandos. Además investigue el uso de las llamadas al sistema fork(), execv(), getpid(), getppid(), waitpid() y wait() en la ayuda en línea, mencione que otras funciones similares a execv() existen, reporte sus observaciones.

Comando ps

El comando ps se utiliza para mostrar información sobre los procesos en ejecución en el sistema. Este comando puede usarse para monitorear el estado de los procesos, ver quién los está ejecutando y entender el uso de recursos en tiempo real.

Algunas opciones comunes de ps incluyen:

- **-e:** Muestra todos los procesos en el sistema.
- **-f:** Muestra los procesos en un formato completo, con información detallada como UID, PID, PPID, entre otros.
- **-a:** Incluye todos los procesos que no pertenecen a la sesión actual.
- **-u [usuario]:** Muestra los procesos del usuario especificado.
- **-l:** Muestra los procesos con un formato extendido y de bajo nivel.
- **-p [PID]:** Muestra únicamente el proceso con el PID especificado.
- **--sort:** Ordena la salida de ps según criterios específicos (ej., -sort=-pcpu para ordenar por uso de CPU de mayor a menor).
- **-o:** Permite personalizar las columnas a mostrar en la salida, como ps -e -o pid,comm,%cpu.

Ejemplo: `ps -ef` muestra todos los procesos en un formato detallado para cada proceso en el sistema.

Comando pstree

El comando `pstree` muestra la jerarquía de procesos en forma de árbol, permitiendo ver la relación padre-hijo entre los procesos.

Opciones comunes de `pstree`:

- **-p**: Muestra el PID junto al nombre de cada proceso.
- **-a**: Muestra los argumentos de línea de comandos para cada proceso.
- **-u**: Muestra el usuario que ha iniciado cada proceso.
- **-n**: Ordena la salida en función del PID.
- **-A**: Usa caracteres ASCII para crear el árbol, útil si el terminal no soporta gráficos extendidos.
- **-c**: Expande todas las ramas del árbol, sin agrupar procesos similares.

Ejemplo: `pstree -p -a` muestra la jerarquía de procesos, los PID, y los argumentos con los que cada proceso fue lanzado.

Llamadas al Sistema en Linux

Las llamadas al sistema permiten a los programas interactuar directamente con el núcleo del sistema operativo para realizar operaciones de bajo nivel como creación de procesos, gestión de recursos, etc.

fork()

Crea un nuevo proceso duplicando el proceso que llama. El proceso original se denomina "padre" y el nuevo "hijo". Esta función regresa 0 al proceso hijo y el PID del hijo al proceso padre.

execv()

Ejecuta un nuevo programa en el proceso actual. Toma el nombre del archivo ejecutable y un vector de argumentos como parámetros. Cuando se llama `execv()`, el programa actual es reemplazado completamente por el nuevo, y no se regresa al código original.

getpid()

Devuelve el PID del proceso que realiza la llamada. Es útil para identificar el proceso actual dentro de su propio contexto.

getppid()

Devuelve el PID del proceso padre, es decir, el proceso que generó al actual. Es útil para rastrear la jerarquía de procesos.

waitpid()

Permite al proceso padre esperar la finalización de un proceso hijo específico. Puede aceptar opciones para el comportamiento de espera (por ejemplo, WNOHANG para que no bloquee si el proceso hijo no ha terminado).

wait()

Es similar a waitpid(), pero espera la terminación de cualquier proceso hijo y devuelve su PID. Si no hay procesos hijos en ejecución, bloquea el proceso padre hasta que alguno finalice.

Otras funciones similares a execv()

La familia de funciones exec incluye varias variaciones que permiten ejecutar nuevos programas de diferentes maneras. Entre ellas están:

- **execl()**: Similar a execv(), pero recibe los argumentos como una lista de parámetros en lugar de un vector.
- **execvp()**: Busca el ejecutable en los directorios especificados en la variable PATH. Usa un vector de argumentos.
- **execle()**: Similar a execl(), pero también permite especificar el entorno del nuevo programa.
- **execlp()**: Combinación de execl() y execvp(). Los argumentos se pasan como una lista y busca el ejecutable en PATH.
- **execve()**: Similar a execv(), pero permite pasar variables de entorno adicionales al nuevo programa.
- **execvpe()**: Similar a execvp(), pero también permite especificar el entorno del nuevo programa.

3. Capture, compile y ejecute los dos programas de creación de un nuevo proceso por copia exacta de código que a continuación se muestra. Observe su funcionamiento y experimente con el código.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(void){
    int id_proc;
    id_proc = fork();
    if (id_proc == 0)
    {
        printf("Soy el proceso hijo\n");
        exit(0);
    }
    else if (id_proc > 0)
    {
        printf("Soy el proceso padre\n");
        exit(0);
    }
    else
    {
        perror("Error al crear el proceso");
        exit(1);
    }
    return 0;
}
```

```
said@said-VirtualBox:~/Descargas$ ./p1
Soy el proceso padre
Soy el proceso hijo
```



```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(void)
{
    int id_proc;
    id_proc=fork();
    if (id_proc == 0)
    {
        printf("Soy el proceso hijo\n");
    }
    else
    {
        printf("Soy el proceso padre\n");
    }
    printf("Mensaje en ambos\n");
    exit(0);
}
```

```
said@said-VirtualBox:~/Descargas$ ./p2
Soy el proceso padre
Mensaje en ambos
Soy el proceso hijo
Mensaje en ambos
```

4. Programe una aplicación que cree el árbol de procesos mostrado en el pizarrón. Para cada uno de los procesos creados (por copia exacta de código) se imprimirá en pantalla el pid de su padre, y cada proceso padre imprimirá los pids de sus hijos creados. Dibuje en papel el árbol creado usando los pids impresos por la aplicación desarrollada.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main(void){
    int id_proc;
    id_proc = fork();
    if(id_proc == 0){
        printf("Soy el proceso main, PID: %d\n", getppid());
        printf("Soy el primer hijo, PID: %d, PID del padre: %d\n", getpid(), getppid());
        id_proc = fork();
        if(id_proc == 0){
            printf("\tSoy el primer nieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            for(int a = 0; a < 5; a++){
                id_proc = fork();
                if(id_proc == 0){
                    if(a == 0){
                        printf("\t\tSoy el primer bisnieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                        for(int b = 0; b < 4; b++){
                            id_proc = fork();
                            if(id_proc == 0){
                                if(b == 0){
                                    printf("\t\t\tSoy el primer tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                                }
                                if(b == 1){
                                    printf("\t\t\tSoy el segundo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                                }
                                if(b == 2){
                                    printf("\t\t\tSoy el tercer tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                                }
                                if(b == 3){
                                    printf("\t\t\tSoy el cuarto tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                                }
                                exit(0);
                            } else {
                                wait(NULL);
                            }
                        }
                    }
                    exit(0);
                }
            }
        }
    }
}
```

```

if(a == 1){
    printf("\t\t\tSoy el segundo bisnieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
    for(int c = 0; c < 2; c++){
        id_proc = fork();
        if(id_proc == 0){
            if(c == 0){
                printf("\t\t\t\tSoy el quinto tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                id_proc = fork();
                if(id_proc == 0){
                    printf("\t\t\t\t\tSoy el primer tataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                    id_proc = fork();
                    if(id_proc == 0){
                        printf("\t\t\t\t\t\tSoy el primer tataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                        exit(0);
                    }
                    wait(NULL);
                    exit(0);
                }
                wait(NULL);
                exit(0);
            }
        }
        if(c == 1){
            printf("\t\t\t\tSoy el sexto tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            id_proc = fork();
            if(id_proc == 0){
                printf("\t\t\t\t\tSoy el segundo tataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                id_proc = fork();
                if(id_proc == 0){
                    printf("\t\t\t\t\t\tSoy el segundo tataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                    id_proc = fork();
                    if(id_proc == 0){
                        printf("\t\t\t\t\t\t\tSoy el primer tataratataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                        exit(0);
                    }
                    wait(NULL);
                    exit(0);
                }
            }
        }
    }
}

```

```

        }
        wait(NULL);
        exit(0);
    }
    wait(NULL);
    exit(0);
}
} else {
    wait(NULL);
}
}
exit(0);
}
}
if(a == 2){
    printf("\t\t\tSoy el tercer bisnieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
    for(int d = 0; d < 3; d++){
        id_proc = fork();
        if(id_proc == 0){
            if(d == 0){
                printf("\t\t\t\tSoy el septimo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            if(d == 1){
                printf("\t\t\t\tSoy el octavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            if(d == 2){
                printf("\t\t\t\tSoy el noveno tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            exit(0);
        } else {
            wait(NULL);
        }
    }
    exit(0);
}
}
if(a == 3){
    printf("\t\t\tSoy el cuarto bisnieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
}

```

```

for(int e = 0; e < 2; e++){
    id_proc = fork();
    if(id_proc == 0){
        if(e == 0){
            printf("\t\t\tSoy el decimo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            id_proc = fork();
            if(id_proc == 0){
                printf("\t\t\t\tSoy el tercer tataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                id_proc = fork();
                if(id_proc == 0){
                    printf("\t\t\t\t\tSoy el tercer tataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                    id_proc = fork();
                    if(id_proc == 0){
                        printf("\t\t\t\t\t\tSoy el segundo tataratataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                        exit(0);
                    }
                    wait(NULL);
                    exit(0);
                }
                wait(NULL);
                exit(0);
            }
            wait(NULL);
            exit(0);
        }
        if(e == 1){
            printf("\t\t\tSoy el onceavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            id_proc = fork();
            if(id_proc == 0){
                printf("\t\t\t\tSoy el cuarto tataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                id_proc = fork();
                if(id_proc == 0){
                    printf("\t\t\t\t\tSoy el cuarto tataratataratataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
                    exit(0);
                }
                wait(NULL);
            }
        }
    }
}

```

```

        }
        wait(NULL);
        exit(0);
    }
    } else {
        wait(NULL);
    }
}
exit(0);
}
if(a == 4){
    printf("\t\tSoy el quinto bisnieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
    for(int f = 0; f < 4; f++){
        id_proc = fork();
        if(id_proc == 0){
            if(f == 0){
                printf("\t\t\tSoy el doceavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            if(f == 1){
                printf("\t\t\t\tSoy el treceavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            if(f == 2){
                printf("\t\t\t\t\tSoy el catorceavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            if(f == 3){
                printf("\t\t\t\t\t\tSoy el quinceavo tataranieto, PID: %d, PID del padre: %d\n", getpid(), getppid());
            }
            exit(0);
        } else {
            wait(NULL);
        }
    }
    exit(0);
}
exit(0);
}

```

```

    }
    wait(NULL);
}
exit(0);
}
wait(NULL);
exit(0);
}
wait(NULL);
return 0;
}

```

```

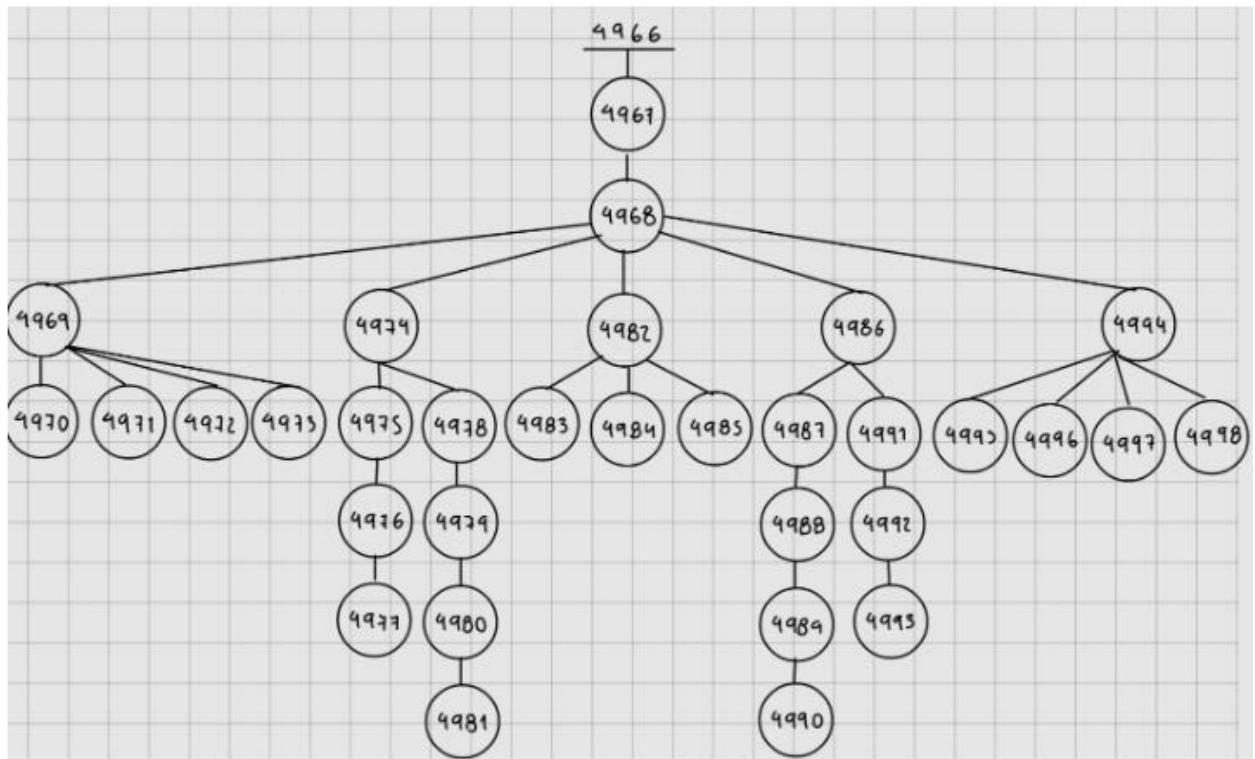
said@said-VirtualBox:~/Descargas$ gcc arbol.c -o arbol
said@said-VirtualBox:~/Descargas$ ./arbol
Soy el proceso main, PID: 4966
Soy el primer hijo, PID: 4967, PID del padre: 4966
  Soy el primer nieto, PID: 4968, PID del padre: 4967
    Soy el primer bisnieto, PID: 4969, PID del padre: 4968
      Soy el primer tataranieto, PID: 4970, PID del padre: 4969
      Soy el segundo tataranieto, PID: 4971, PID del padre: 4969
      Soy el tercer tataranieto, PID: 4972, PID del padre: 4969
      Soy el cuarto tataranieto, PID: 4973, PID del padre: 4969
    Soy el segundo bisnieto, PID: 4974, PID del padre: 4968
      Soy el quinto tataranieto, PID: 4975, PID del padre: 4974
        Soy el primer tataratataranieto, PID: 4976, PID del padre: 4975
          Soy el primer tataratataratataranieto, PID: 4977, PID del padre: 4976
        Soy el sexto tataranieto, PID: 4978, PID del padre: 4974
          Soy el segundo tataratataranieto, PID: 4979, PID del padre: 4978
            Soy el segundo tataratataratataranieto, PID: 4980, PID del padre: 4979
              Soy el primer tataratataratataratataranieto, PID: 4981, PID del padre: 4
980
        Soy el tercer bisnieto, PID: 4982, PID del padre: 4968
          Soy el septimo tataranieto, PID: 4983, PID del padre: 4982
          Soy el octavo tataranieto, PID: 4984, PID del padre: 4982
          Soy el noveno tataranieto, PID: 4985, PID del padre: 4982
        Soy el cuarto bisnieto, PID: 4986, PID del padre: 4968
          Soy el decimo tataranieto, PID: 4987, PID del padre: 4986
            Soy el tercer tataratataranieto, PID: 4988, PID del padre: 4987
              Soy el tercer tataratataratataranieto, PID: 4989, PID del padre: 4988
                Soy el segundo tataratataratataratataranieto, PID: 4990, PID del padre:
4989
                Soy el onceavo tataranieto, PID: 4991, PID del padre: 4986

```

```

                Soy el cuarto tataratataranieto, PID: 4992, PID del padre: 4991
                  Soy el cuarto tataratataratataranieto, PID: 4993, PID del padre: 4992
                Soy el quinto bisnieto, PID: 4994, PID del padre: 4968
                  Soy el doceavo tataranieto, PID: 4995, PID del padre: 4994
                  Soy el treceavo tataranieto, PID: 4996, PID del padre: 4994
                  Soy el catorceavo tataranieto, PID: 4997, PID del padre: 4994
                  Soy el quinceavo tataranieto, PID: 4998, PID del padre: 4994
said@said-VirtualBox:~/Descargas$ █

```



5. Programe una aplicación que cree seis procesos (por copia exacta de código). El primer proceso se encargará de realizar la suma de dos matrices de 10x10 elementos tipo entero, el segundo proceso realizará la resta sobre esas mismas matrices, el tercer proceso realizará la multiplicación de las matrices, el cuarto proceso obtendrá las transpuestas de cada matriz y el quinto proceso obtendrá las matrices inversas. Cada uno de estos procesos escribirá un archivo con los resultados de la operación que realizó. El sexto proceso leerán los archivos de resultados y los mostrará en pantalla cada uno de ellos. Programe la misma aplicación sin la creación de procesos, es decir de forma secuencial. Obtenga los tiempos de ejecución de cada una de las aplicaciones programadas, compare estos tiempos y dé sus observaciones.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>

#define N 10
void imprimirMatriz(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}
```

```

void leerYMostrarMatriz(char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "r");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    double valor;
    int cont=0;
    while (fscanf(archivo, "%lf", &valor) != EOF) {
        if(cont == 10){
            printf("\n");
            cont=0;
        }
        printf("%.2f ", valor);
        cont ++;
    }
    printf("\n");
    fclose(archivo);
}

void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}

```



```

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

```

```

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }

    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

int main() {
    srand(time(NULL));
    int A[N][N], B[N][N];
    double resultado[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    pid_t pid;
    for (int i = 1; i <= 5; i++) {
        pid = fork();
        if (pid == 0) {
            switch (i) {
                case 1:
                    sumarMatrices(A, B, resultado);
                    escribirMatriz("suma.txt", resultado);
                    break;
            }
        }
    }
}

```

```

        case 2:
            restarMatrices(A, B, resultado);
            escribirMatriz("resta.txt", resultado);
            break;
        case 3:
            multiplicarMatrices(A, B, resultado);
            escribirMatriz("multiplicacion.txt", resultado);
            break;
        case 4:
            transponerMatriz(A, resultado);
            escribirMatriz("transpuesta_A.txt", resultado);
            transponerMatriz(B, resultado);
            escribirMatriz("transpuesta_B.txt", resultado);
            break;
        case 5:
            double Inversauno[N][N];
            double Inversados[N][N];
            InversaMatriz(A, Inversauno);
            InversaMatriz(B, Inversados);
            escribirMatriz("inversa_A.txt", Inversauno);
            escribirMatriz("inversa_B.txt", Inversados);
            break;
    }
    exit(0);
}

}

for (int i = 0; i < 5; i++) {
    wait(NULL);
}

pid = fork();
if (pid == 0) {
    printf("\nResultados de la suma:\n");
    leerYMostrarMatriz("suma.txt");
    printf("\nResultados de la resta:\n");
    leerYMostrarMatriz("resta.txt");
    printf("\nResultados de la multiplicación:\n");
    leerYMostrarMatriz("multiplicacion.txt");
}

```

```

printf("\nTranspuesta de la matriz A:\n");
leerYMostrarMatriz("transpuesta_A.txt");
printf("\nTranspuesta de la matriz B:\n");
leerYMostrarMatriz("transpuesta_B.txt");
printf("\nInversa de la matriz A:\n");
leerYMostrarMatriz("inversa_A.txt");
printf("\nInversa de la matriz B:\n");
leerYMostrarMatriz("inversa_B.txt");
exit(0);
}
wait(NULL);
return 0;
}

```



inversa_B.txt



inversa_A.txt



transpuesta_B.txt



transpuesta_A.txt



suma.txt



multiplicacion.txt

said@said-VirtualBox:~/Descargas\$ time ./mat

Matriz A:

96	60	80	69	26	33	19	8	2	44
27	13	77	83	64	60	56	97	37	96
37	53	97	43	91	59	17	33	21	80
56	9	91	46	70	91	36	63	72	63
24	79	26	67	59	98	18	25	78	37
45	53	37	45	87	70	96	85	93	68
65	32	3	58	95	60	89	52	95	67
70	25	17	86	27	95	16	7	16	66
26	82	72	5	29	13	75	30	14	34
68	32	21	70	36	80	94	42	94	98

Matriz B:

15	72	52	27	73	24	1	20	63	94
64	49	83	20	41	59	19	2	65	63
28	23	62	65	53	17	38	60	15	85
5	78	5	28	7	74	44	99	72	62
21	7	78	47	47	81	73	9	71	13
27	47	15	8	7	78	15	21	18	52
31	83	23	47	95	5	13	92	97	48
36	65	9	77	19	58	93	19	65	77
93	14	33	66	38	99	38	58	82	7
28	84	5	3	1	62	61	65	14	89

Resultados de la suma:

```
111.00 132.00 132.00 96.00 99.00 57.00 20.00 28.00 65.00 138.00
91.00 62.00 160.00 103.00 105.00 119.00 75.00 99.00 102.00 159.00
65.00 76.00 159.00 108.00 144.00 76.00 55.00 93.00 36.00 165.00
61.00 87.00 96.00 74.00 77.00 165.00 80.00 162.00 144.00 125.00
45.00 86.00 104.00 114.00 106.00 179.00 91.00 34.00 149.00 50.00
72.00 100.00 52.00 53.00 94.00 148.00 111.00 106.00 111.00 120.00
96.00 115.00 26.00 105.00 190.00 65.00 102.00 144.00 192.00 115.00
106.00 90.00 26.00 163.00 46.00 153.00 109.00 26.00 81.00 143.00
119.00 96.00 105.00 71.00 67.00 112.00 113.00 88.00 96.00 41.00
96.00 116.00 26.00 73.00 37.00 142.00 155.00 107.00 108.00 187.00
```

Resultados de la resta:

```
81.00 -12.00 28.00 42.00 -47.00 9.00 18.00 -12.00 -61.00 -50.00
-37.00 -36.00 -6.00 63.00 23.00 1.00 37.00 95.00 -28.00 33.00
9.00 30.00 35.00 -22.00 38.00 42.00 -21.00 -27.00 6.00 -5.00
51.00 -69.00 86.00 18.00 63.00 17.00 -8.00 -36.00 0.00 1.00
3.00 72.00 -52.00 20.00 12.00 17.00 -55.00 16.00 7.00 24.00
18.00 6.00 22.00 37.00 80.00 -8.00 81.00 64.00 75.00 16.00
34.00 -51.00 -20.00 11.00 0.00 55.00 76.00 -40.00 -2.00 19.00
34.00 -40.00 8.00 9.00 8.00 37.00 -77.00 -12.00 -49.00 -11.00
-67.00 68.00 39.00 -61.00 -9.00 -86.00 37.00 -28.00 -68.00 27.00
40.00 -52.00 16.00 67.00 35.00 18.00 33.00 -23.00 80.00 9.00
```

Resultados de la multiplicación:

```
11597.00 24628.00 18595.00 14183.00 17721.00 20475.00 13456.00 19474.00 21699.00 31394.00
18129.00 33629.00 17426.00 24637.00 19259.00 34251.00 29435.00 30620.00 31416.00 37960.00
16290.00 24826.00 22316.00 19283.00 18126.00 29857.00 23118.00 21590.00 24486.00 32603.00
19965.00 28304.00 20442.00 24397.00 20945.00 34462.00 25090.00 26473.00 29771.00 34750.00
20112.00 23761.00 19222.00 17381.00 15483.00 34616.00 19018.00 20480.00 28099.00 26088.00
25634.00 34604.00 23476.00 27988.00 26650.00 39506.00 28674.00 30195.00 40927.00 36131.00
22354.00 32051.00 20807.00 23817.00 24627.00 37186.00 24864.00 28442.00 38962.00 31236.00
10772.00 25569.00 12019.00 10477.00 11899.00 25567.00 13864.00 20045.00 20352.00 27516.00
14298.00 19977.00 17731.00 15490.00 18826.00 16028.00 13223.00 16525.00 21550.00 24127.00
22834.00 36499.00 17984.00 22283.00 23529.00 36501.00 23060.00 32886.00 36643.00 36287.00
```

Transpuesta de la matriz A:

```
96.00 27.00 37.00 56.00 24.00 45.00 65.00 70.00 26.00 68.00
60.00 13.00 53.00 9.00 79.00 53.00 32.00 25.00 82.00 32.00
80.00 77.00 97.00 91.00 26.00 37.00 3.00 17.00 72.00 21.00
69.00 83.00 43.00 46.00 67.00 45.00 58.00 86.00 5.00 70.00
26.00 64.00 91.00 70.00 59.00 87.00 95.00 27.00 29.00 36.00
33.00 60.00 59.00 91.00 98.00 70.00 60.00 95.00 13.00 80.00
19.00 56.00 17.00 36.00 18.00 96.00 89.00 16.00 75.00 94.00
8.00 97.00 33.00 63.00 25.00 85.00 52.00 7.00 30.00 42.00
2.00 37.00 21.00 72.00 78.00 93.00 95.00 16.00 14.00 94.00
44.00 96.00 80.00 63.00 37.00 68.00 67.00 66.00 34.00 98.00
```

Transpuesta de la matriz B:

```
15.00 64.00 28.00 5.00 21.00 27.00 31.00 36.00 93.00 28.00
72.00 49.00 23.00 78.00 7.00 47.00 83.00 65.00 14.00 84.00
52.00 83.00 62.00 5.00 78.00 15.00 23.00 9.00 33.00 5.00
27.00 20.00 65.00 28.00 47.00 8.00 47.00 77.00 66.00 3.00
73.00 41.00 53.00 7.00 47.00 7.00 95.00 19.00 38.00 1.00
24.00 59.00 17.00 74.00 81.00 78.00 5.00 58.00 99.00 62.00
1.00 19.00 38.00 44.00 73.00 15.00 13.00 93.00 38.00 61.00
20.00 2.00 60.00 99.00 9.00 21.00 92.00 19.00 58.00 65.00
63.00 65.00 15.00 72.00 71.00 18.00 97.00 65.00 82.00 14.00
94.00 63.00 85.00 62.00 13.00 52.00 48.00 77.00 7.00 89.00
```

Inversa de la matriz A:

```
-1.36 -0.51 -0.78 0.04 -0.27 0.02 -0.50 -0.33 1.34 0.76
0.26 -0.89 -0.42 0.91 0.44 0.27 1.30 0.55 -0.49 -1.33
-0.73 1.13 -0.65 -1.02 0.63 0.70 0.64 -0.85 -1.02 -0.85
1.10 0.74 0.48 -0.38 1.17 -0.14 -0.64 0.65 -1.30 -0.15
1.10 0.30 1.11 1.08 -0.49 -0.51 0.70 0.22 0.89 1.15
-0.94 -0.94 -1.37 0.24 -0.12 -0.61 -1.26 1.19 0.94 -0.90
1.34 1.32 0.54 0.97 0.49 -1.14 0.16 -0.51 0.06 -1.25
0.22 -0.54 1.27 -1.38 1.24 -0.16 -1.00 -0.37 -1.29 0.15
0.02 0.48 -0.88 0.51 -0.41 0.51 -1.05 -0.22 -0.42 0.21
-0.12 0.81 -0.35 1.33 1.39 0.48 -0.34 -1.11 1.26 -0.19
```

Inversa de la matriz B:

```
-0.19 -0.37 1.15 0.43 1.09 0.99 -0.22 -0.21 -0.82 0.14
-0.04 0.49 -1.30 -0.05 -0.12 -0.09 -1.02 0.24 -0.30 0.10
0.99 1.10 0.55 0.01 -0.31 -0.16 -0.32 0.93 -1.29 1.11
-1.34 -1.14 0.50 -0.11 -0.67 -0.97 -1.16 1.13 -0.70 -0.03
-0.06 -0.13 0.35 -0.46 0.41 0.36 -0.32 0.94 -0.20 1.03
-0.77 -0.45 1.15 -0.33 0.67 0.02 -0.55 0.65 -0.65 1.11
1.14 0.54 1.02 -0.44 -1.01 0.90 -0.27 -0.58 1.12 0.30
-0.77 -1.30 -1.05 1.11 0.59 -1.28 -1.02 -0.22 0.41 0.85
-0.19 -0.52 1.31 -1.27 -1.24 -0.70 0.42 0.17 0.01 -0.24
0.28 1.22 1.00 1.09 -0.33 0.99 0.71 -0.88 -0.49 -0.54
```

Tiempo:

```
real    0m2,661s
user    0m2,646s
sys     0m0,016s
```


Versión sin procesos:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>

#define N 10
void imprimirMatriz(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}
```



```

void leerYMostrarMatriz(char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "r");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    double valor;
    int cont=0;
    while (fscanf(archivo, "%lf", &valor) != EOF) {
        if(cont == 10){
            printf("\n");
            cont=0;
        }
        printf("%.2f ", valor);
        cont ++;
    }
    printf("\n");
    fclose(archivo);
}

void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}

```

```

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

```

```

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```

```

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }

    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

int main() {
    srand(time(NULL));
    int A[N][N], B[N][N];
    double resultado[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }

    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    sumarMatrices(A, B, resultado);
    escribirMatriz("suma.txt", resultado);
    restarMatrices(A, B, resultado);
    escribirMatriz("resta.txt", resultado);
    multiplicarMatrices(A, B, resultado);
    escribirMatriz("multiplicacion.txt", resultado);
    transponerMatriz(A, resultado);
    escribirMatriz("transpuesta_A.txt", resultado);
    transponerMatriz(B, resultado);
    escribirMatriz("transpuesta_B.txt", resultado);
    double Inversauno[N][N];
    double Inversados[N][N];
}

```

```
InversaMatriz(A, Inversauno);
InversaMatriz(B, Inversados);
escribirMatriz("inversa_A.txt", Inversauno);
escribirMatriz("inversa_B.txt", Inversados);
printf("\nResultados de la suma:\n");
leerYMostrarMatriz("suma.txt");
printf("\nResultados de la resta:\n");
leerYMostrarMatriz("resta.txt");
printf("\nResultados de la multiplicación:\n");
leerYMostrarMatriz("multiplicacion.txt");
printf("\nTranspuesta de la matriz A:\n");
leerYMostrarMatriz("transpuesta_A.txt");
printf("\nTranspuesta de la matriz B:\n");
leerYMostrarMatriz("transpuesta_B.txt");
printf("\nInversa de la matriz A:\n");
leerYMostrarMatriz("inversa_A.txt");
printf("\nInversa de la matriz B:\n");
leerYMostrarMatriz("inversa_B.txt");
return 0;
```

```
}
```

```
said@said-VirtualBox:~/Descargas$ time ./mat1
```

```
Matriz A:
```

75	87	90	23	34	69	47	49	39	85
92	12	99	16	23	7	82	57	12	67
86	82	16	34	90	38	6	92	11	34
88	64	68	72	32	25	72	15	71	76
67	7	97	77	73	74	19	61	23	66
37	38	65	63	35	61	51	40	100	20
60	99	97	99	58	6	55	37	29	20
71	98	82	34	87	86	13	92	75	27
19	61	9	26	29	69	61	87	36	56
40	68	57	61	14	97	87	41	20	0

```
Matriz B:
```

2	60	53	29	40	70	15	1	26	51
55	45	95	64	80	98	60	37	46	12
1	32	98	21	85	81	50	4	8	0
8	35	20	47	4	17	7	88	15	26
80	73	99	74	11	26	37	53	33	55
92	9	19	35	4	68	34	47	83	72
60	79	25	86	64	62	10	86	66	30
48	21	3	47	54	46	50	76	90	100
33	41	98	38	13	100	32	8	20	22
7	19	25	98	54	34	23	62	67	50

Resultados de la suma:

```
77.00 147.00 143.00 52.00 74.00 139.00 62.00 50.00 65.00 136.00
147.00 57.00 194.00 80.00 103.00 105.00 142.00 94.00 58.00 79.00
87.00 114.00 114.00 55.00 175.00 119.00 56.00 96.00 19.00 34.00
96.00 99.00 88.00 119.00 36.00 42.00 79.00 103.00 86.00 102.00
147.00 80.00 196.00 151.00 84.00 100.00 56.00 114.00 56.00 121.00
129.00 47.00 84.00 98.00 39.00 129.00 85.00 87.00 183.00 92.00
120.00 178.00 122.00 185.00 122.00 68.00 65.00 123.00 95.00 50.00
119.00 119.00 85.00 81.00 141.00 132.00 63.00 168.00 165.00 127.00
52.00 102.00 107.00 64.00 42.00 169.00 93.00 95.00 56.00 78.00
47.00 87.00 82.00 159.00 68.00 131.00 110.00 103.00 87.00 50.00
```

Resultados de la resta:

```
73.00 27.00 37.00 -6.00 -6.00 -1.00 32.00 48.00 13.00 34.00
37.00 -33.00 4.00 -48.00 -57.00 -91.00 22.00 20.00 -34.00 55.00
85.00 50.00 -82.00 13.00 5.00 -43.00 -44.00 88.00 3.00 34.00
80.00 29.00 48.00 25.00 28.00 8.00 65.00 -73.00 56.00 50.00
-13.00 -66.00 -2.00 3.00 62.00 48.00 -18.00 8.00 -10.00 11.00
-55.00 29.00 46.00 28.00 31.00 -7.00 17.00 -7.00 17.00 -52.00
0.00 20.00 72.00 13.00 -6.00 -56.00 45.00 -49.00 -37.00 -10.00
23.00 77.00 79.00 -13.00 33.00 40.00 -37.00 16.00 -15.00 -73.00
-14.00 20.00 -89.00 -12.00 16.00 -31.00 29.00 79.00 16.00 34.00
33.00 49.00 32.00 -37.00 -40.00 63.00 64.00 -21.00 -47.00 -50.00
```

Resultados de la multiplicación:

```
21331.00 23159.00 33466.00 31802.00 29103.00 38991.00 20733.00 24071.00 27853.00 23723.00
12076.00 20970.00 23520.00 24967.00 25500.00 28165.00 13846.00 19522.00 20587.00 18795.00
21043.00 20967.00 26582.00 26256.00 19969.00 27714.00 17664.00 22436.00 23944.00 25262.00
17115.00 25775.00 33194.00 31746.00 25605.00 36534.00 16588.00 24547.00 22601.00 19710.00
19169.00 21108.00 28457.00 27882.00 21265.00 30000.00 17525.00 25383.00 24172.00 25322.00
19565.00 20668.00 29520.00 24582.00 18730.00 33390.00 16065.00 21435.00 20699.00 19028.00
17819.00 25603.00 34755.00 28799.00 26598.00 34050.00 18521.00 25193.00 19667.00 17432.00
28618.00 26156.00 40662.00 32037.00 27320.00 43360.00 25177.00 26054.00 29976.00 29248.00
21694.00 17007.00 19100.00 26618.00 19198.00 27213.00 15396.00 24998.00 26774.00 23062.00
22257.00 19868.00 22873.00 24176.00 20713.00 31358.00 15333.00 24211.00 23884.00 19346.00
```

Transpuesta de la matriz A:

```
75.00 92.00 86.00 88.00 67.00 37.00 60.00 71.00 19.00 40.00
87.00 12.00 82.00 64.00 7.00 38.00 99.00 98.00 61.00 68.00
90.00 99.00 16.00 68.00 97.00 65.00 97.00 82.00 9.00 57.00
23.00 16.00 34.00 72.00 77.00 63.00 99.00 34.00 26.00 61.00
34.00 23.00 90.00 32.00 73.00 35.00 58.00 87.00 29.00 14.00
69.00 7.00 38.00 25.00 74.00 61.00 6.00 86.00 69.00 97.00
47.00 82.00 6.00 72.00 19.00 51.00 55.00 13.00 61.00 87.00
49.00 57.00 92.00 15.00 61.00 40.00 37.00 92.00 87.00 41.00
39.00 12.00 11.00 71.00 23.00 100.00 29.00 75.00 36.00 20.00
85.00 67.00 34.00 76.00 66.00 20.00 20.00 27.00 56.00 0.00
```

Transpuesta de la matriz B:

```
2.00 55.00 1.00 8.00 80.00 92.00 60.00 48.00 33.00 7.00
60.00 45.00 32.00 35.00 73.00 9.00 79.00 21.00 41.00 19.00
53.00 95.00 98.00 20.00 99.00 19.00 25.00 3.00 98.00 25.00
29.00 64.00 21.00 47.00 74.00 35.00 86.00 47.00 38.00 98.00
40.00 80.00 85.00 4.00 11.00 4.00 64.00 54.00 13.00 54.00
70.00 98.00 81.00 17.00 26.00 68.00 62.00 46.00 100.00 34.00
15.00 60.00 50.00 7.00 37.00 34.00 10.00 50.00 32.00 23.00
1.00 37.00 4.00 88.00 53.00 47.00 86.00 76.00 8.00 62.00
26.00 46.00 8.00 15.00 33.00 83.00 66.00 90.00 20.00 67.00
51.00 12.00 0.00 26.00 55.00 72.00 30.00 100.00 22.00 50.00
```

Inversa de la matriz A:

```
0.78 0.26 -0.59 0.50 -1.22 0.39 0.31 -0.10 1.70 0.35
-1.31 -0.92 0.72 -1.61 -1.72 -1.55 -1.85 1.78 0.06 -1.70
1.57 0.32 0.82 0.66 -1.16 -0.89 0.73 1.58 1.78 0.07
-1.12 1.03 -0.51 -1.10 0.69 -1.54 0.87 1.47 0.91 1.89
1.28 1.82 -0.56 1.38 0.09 1.40 1.21 1.66 0.98 -1.09
1.23 -0.74 0.11 0.39 0.79 -0.73 0.44 -0.58 0.31 -0.67
-1.68 -1.31 -0.16 1.43 0.24 1.27 -1.42 1.30 1.84 -0.04
0.98 0.60 1.22 -1.80 1.04 -0.82 0.27 -0.47 0.77 1.66
-1.07 -0.51 0.65 0.57 0.03 1.87 1.80 0.20 -0.45 -0.21
0.39 -0.85 -0.73 1.26 0.88 0.58 0.67 0.22 0.50 -1.20
```

Inversa de la matriz B:

```
2.69 8.96 12.33 -8.28 -5.06 -6.24 -14.87 10.98 13.88 -5.80
-4.12 -5.08 5.19 -15.05 -6.04 14.96 6.07 0.07 -9.64 4.85
0.61 2.53 10.76 -1.45 -4.33 7.55 -10.47 -2.46 2.96 5.73
9.50 -13.72 -1.72 2.21 -10.36 -13.16 -6.44 6.66 -6.93 -11.96
13.78 14.84 -5.04 7.82 -6.46 13.78 10.13 -5.33 -1.69 -6.76
-13.37 -9.61 -11.35 -6.40 13.83 -4.25 -4.19 10.35 -13.10 -13.11
-5.69 -9.02 -8.70 -4.20 11.99 -11.75 14.37 -5.74 -3.72 12.33
-1.38 -1.64 6.92 3.76 8.57 4.51 -0.66 -1.91 8.57 -11.41
1.39 -9.17 -5.61 11.42 11.00 1.69 1.92 14.78 -11.58 7.82
4.89 -4.55 10.38 -7.98 -10.25 9.40 1.77 3.14 7.62 1.81
```

Tiempo:

```
real    0m4,387s
user    0m4,157s
sys     0m0,213s
```


Observaciones:

La velocidad de ejecución de un programa puede verse afectada significativamente por la creación de procesos.

Programa sin creación de procesos

- En un programa sin creación de procesos, todo el código se ejecuta en un solo hilo o proceso. Esto significa que las instrucciones se ejecutan secuencialmente, una tras otra.
- En general, los programas sin procesos son más simples y fáciles de entender, pero pueden ser menos eficientes en términos de rendimiento.

Programa con creación de procesos

- Cuando se crean procesos adicionales, el programa se divide en múltiples hilos o procesos independientes.
- Cada proceso tiene su propio espacio de memoria y puede ejecutarse en paralelo con otros procesos.
- Esto permite aprovechar mejor los recursos de la CPU y, en algunos casos, acelerar la ejecución. • Sin embargo, la creación de procesos también introduce cierta sobrecarga debido a la asignación de memoria y la comunicación entre procesos.

Factores de Velocidad entre Programas con Procesos vs. sin Procesos

1. Sobrecarga de Creación de Procesos:

- En un programa que utiliza procesos, cada llamada a `fork()` introduce una cierta sobrecarga, ya que el sistema operativo necesita crear un nuevo proceso, asignarle memoria y recursos.
- La creación de procesos es generalmente costosa en términos de tiempo y recursos, lo que puede hacer que un programa con múltiples procesos sea más lento, especialmente para tareas muy cortas o simples.

2. Ejecutar Tareas en Paralelo:

- Si el programa con procesos está diseñado para aprovechar tareas independientes y paralelas (como cálculos en paralelo), puede ser más rápido, ya que múltiples procesos pueden ejecutarse al mismo tiempo en diferentes núcleos de CPU.
- Para tareas que requieren mucho procesamiento, dividir las en varios procesos puede reducir el tiempo total de ejecución, siempre que el hardware lo soporte.

3. Comunicación entre Procesos:

- Cuando se utiliza `fork()`, cada proceso hijo tiene su propio espacio de memoria, aislado del proceso padre. La comunicación entre procesos (IPC) puede ser necesaria, y establecer estos canales (como pipes, sockets o señales) consume recursos y puede ralentizar el programa.

4. Consumo de Memoria:

- Cada proceso consume una cierta cantidad de memoria, lo cual puede ser ineficiente comparado con una versión sin procesos. En un sistema con recursos limitados, ejecutar muchos procesos a la vez podría llevar a problemas de rendimiento debido al uso excesivo de memoria.

6. Capture, compile y ejecute el siguiente programa de creación de un nuevo proceso con sustitución de un nuevo código, así como el programa que será el nuevo código a ejecutar. Observe su funcionamiento y experimente con el código.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;
    char *argv[3];
    argv[0]="p4";
    argv[1]="Desde el Hijo";
    argv[2]=NULL;
    if((pid=fork())==0)
    {
        printf("Soy el hijo ejecutando: %s\n", argv[0]);
        execv(argv[0],argv);
    }
    else
    {
        wait(0);
        printf("Soy el Padre\n");
        exit(0);
    }
}
```

```
/* hola.c Programa que será invocado */  
#include<stdio.h>  
int main(int argc, char *argv[])  
{  
    char mensaje[100];  
    strcpy(mensaje,"Hola Mundo ");  
    strcat(mensaje,argv[1]);  
    printf("%s\n",mensaje);  
    exit(0);  
}
```

```
said@said-VirtualBox:~/Descargas$ ./p3  
Soy el hijo ejecutando: hola  
Hola Mundo Desde el Hijo  
Soy el Padre
```

Observaciones

En el marco de nuestras pruebas, se llevó a cabo la captura, compilación y ejecución de un programa diseñado para experimentar con la sustitución de código. Este proceso permite que el programa original reemplace su propio código en ejecución con el código de una ruta especificada, al cual se le pueden pasar ciertos parámetros.

Durante la ejecución, se observó que, tras la sustitución, el nuevo código se ejecuta en el mismo espacio de proceso del programa principal. Una vez que el nuevo proceso finaliza su tarea, el control regresa al programa original, que continúa su ejecución hasta completarse.

Las pruebas realizadas confirmaron que el mecanismo de sustitución de código funciona de manera efectiva, permitiendo que un nuevo código se ejecute dentro del contexto del proceso principal sin problemas. Esta funcionalidad ofrece una forma interesante de manejar la ejecución de diferentes tareas dentro de un solo proceso.

7. Programe una aplicación que cree un proceso hijo a partir de un proceso padre, el hijo creado a su vez creará tres procesos hijos más. Cada uno de los tres procesos generados ejecutará tres programas diferentes mediante sustitución de código, el primer programa evaluará una expresión aritmética (está aplicación es la que programó en la práctica 1), el segundo programa cambiará los permisos de un archivo dado (está aplicación es la que programó en la práctica 2), y el tercer programa obtendrá las matrices inversas que programó anteriormente. Observe el funcionamiento de su programa detalladamente y responda la siguiente pregunta ¿es posible un funcionamiento 100% concurrente de su aplicación? Explique porque sí o no es 100% concurrente su aplicación.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid_hijo = fork();
    char *argv[4];
    if (pid_hijo < 0) {
        perror("Error al crear el proceso hijo");
        exit(EXIT_FAILURE);
    }
    if (pid_hijo == 0) {
        printf("Soy el Hijo\n");
        for (int i = 0; i < 3; i++) {
            pid_t pid_nieto = fork();
            if (pid_nieto < 0) {
                perror("Error al crear el proceso nieto");
                exit(EXIT_FAILURE);
            }
            printf("Soy el nieto %d\n", i+1);
            if (pid_nieto == 0) {
                switch (i) {
                    case 0:
                        printf("\tSoy el primer proceso nieto\n");
                        argv[0] = "./exp";
                        argv[1] = "20+15*3-5";
                        argv[2] = NULL;
                        execv(argv[0], argv);
                        break;
                    case 1:
                        printf("\tSoy el segundo proceso nieto\n");
                        argv[0] = "./per";
                        argv[1] = "permisos.txt";
                        argv[2] = "777";
                        argv[3] = NULL;
                        execv(argv[0], argv);
                        break;
                }
            }
        }
    }
}
```

```

        case 2:
            printf("\tSoy el tercer proceso nieto\n");
            argv[0] = "./inv";
            argv[1] = NULL;
            execv(argv[0], argv);
            break;
    }
    perror("Error al ejecutar el programa");
    exit(EXIT_FAILURE);
}
for (int i = 0; i < 3; i++) {
    wait(NULL);
}
exit(EXIT_SUCCESS);
}
wait(NULL);
printf("\nEl proceso padre ha terminado.\n");
return 0;
}

```

said@said-VirtualBox:~/Descargas\$./main

Soy el Hijo
 Soy el nieto 1
 Soy el nieto 2
 Soy el nieto 3
 Soy el nieto 1
 Soy el primer proceso nieto
 Soy el nieto 3
 Soy el tercer proceso nieto
 El resultado de la expresión es: 60.000000

Matriz A:

7	99	68	43	15	10	45	0	33	81
21	50	53	34	64	89	0	64	22	10
63	51	62	83	89	15	100	1	12	73
60	80	15	28	16	41	25	20	10	47
66	40	93	50	9	81	17	96	70	1
28	98	68	72	20	8	13	84	22	10
56	34	84	37	53	5	13	66	87	91
88	24	13	73	41	33	40	35	63	43
42	28	84	69	63	42	24	84	6	39
38	9	72	1	82	82	59	46	35	78

Soy el nieto 2
 Soy el segundo proceso nieto
 Permisos cambiados del archivo: permisos.txt

```

Inversa:
0.39    3.46    -2.99    -3.43    -0.11    -1.89    -0.53    -2.74    -2.04    -0.84
0.48    2.85    -2.92    1.98    -2.67    0.74    1.71    -0.30    -2.70    -2.34
1.22    0.62    2.12    -0.19    2.59    -0.48    -0.21    2.69    1.03    3.04
2.23    1.45    3.15    2.10    -0.50    2.03    0.17    0.59    1.29    2.42
2.09    1.22    2.18    2.18    -3.37    -2.50    1.33    -3.31    1.14    -1.45
-1.36    2.78    -1.89    1.46    -0.26    0.87    0.31    -1.38    1.53    0.59
0.21    -3.09    0.34    -3.19    -2.13    1.37    1.64    -2.63    0.89    -0.69
2.22    0.14    -0.17    2.59    0.11    -2.58    -3.39    -2.22    -0.40    -2.01
-2.53    -1.33    2.63    2.04    2.37    1.42    -0.36    3.09    -1.80    -0.51
3.27    -2.01    3.39    -0.85    -3.32    -1.80    0.16    0.04    0.48    -0.46

Matriz B:
42      27      69      43      90      47      75      8      37      54
36      26      82      55      93      37      88      68      74      38
20      8       52      95      64      76      65      18      1      76
28      60      43      3       38      46      59      53      2      65
62      32      8       5       90      36      60      99      23      52
9       19      17      44      85      52      48      5       0      99
58      73      29      53      4       47      2       47      3      73
99      0       75      35      83      97      67      91      72      82
14      56      60      70      60      16      32      3       56      47
38      45      44      0       44      24      9       8       81      98

Inversa:
0.71    -1.46    1.09    -1.51    -0.99    -0.00    0.60    -0.49    0.74    -1.10
-1.02    0.43    0.54    1.09    -1.61    -0.25    -0.07    1.11    -1.10    1.35
0.28    -0.67    -0.77    -1.42    -1.41    -0.98    1.30    -0.39    -1.06    -0.85
-0.71    1.44    -1.32    1.44    -0.46    -0.21    -1.45    -0.74    1.58    1.46
0.03    -0.37    1.04    0.50    0.31    1.35    -0.69    1.36    -1.60    -0.40
-0.20    -0.10    -0.84    -1.10    -0.26    0.87    1.21    1.53    0.42    -0.95
1.10    -0.64    -0.57    0.27    0.93    1.59    0.00    -1.04    -0.96    0.13
-0.15    1.18    1.43    0.01    1.40    1.22    -0.10    -1.22    1.34    1.63
-0.22    0.15    -0.81    -1.20    0.04    1.21    -0.96    0.98    -0.74    -0.73
-0.72    -1.04    1.49    -0.48    1.54    0.10    0.88    1.05    1.05    -1.35

El proceso padre ha terminado.

```

Este programa no es 100 % concurrente porque sigue un flujo secuencial en la creación y ejecución de procesos. Primero, el proceso padre crea un proceso hijo. Luego, el hijo crea tres procesos nietos dentro de un ciclo for, pero cada nieto es creado y ejecutado de manera secuencial, uno tras otro, en lugar de lanzarse todos al mismo tiempo. Después, el proceso hijo utiliza wait para esperar a que terminen sus procesos nietos, lo que introduce bloqueos en la ejecución. Finalmente, el padre espera a que termine el proceso hijo antes de finalizar.

8. Programe la aplicación desarrollada en el punto 5 de la sección de Linux utilizando esta vez la creación de procesos por sustitución de código. Obtenga el tiempo de ejecución de la aplicación programada, compare este tiempo contra el tiempo obtenido del programa de creación de procesos por copia exacta de código, y dé sus observaciones.

Archivo.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]){
    execlp("cat", "cat", "suma.txt", "resta.txt", "multiplicacion.txt", "transpuesta_A.txt",
    "transpuesta_B.txt", "inversa_A.txt", "inversa_B.txt", NULL);
}
```

Inversa.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>

#define N 10
void extraer(int matriz[N][N], char *str){
    char *token = strtok(str, " ");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = atoi(token);
            token = strtok(NULL, " ");
        }
    }
}

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}
```

```

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

```



```

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Inversa\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\tRealizo la inversa de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double Inversa1[N][N];
    double Inversa2[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    InversaMatriz(Matriz1, Inversa1);
    InversaMatriz(Matriz2, Inversa2);
    escribirMatriz("inversa_A.txt", Inversa1);
    escribirMatriz("inversa_B.txt", Inversa2);
    return 0;
}

```

Multiplicación.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>

#define N 10

void extraer(int matriz[N][N], char *str){
    char *token = strtok(str, " ");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = atoi(token);
            token = strtok(NULL, " ");
        }
    }
}

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

```
void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Multiplicacion\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\tRealizo la multiplicacion de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    multiplicarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("multiplicacion.txt", resultado);
    return 0;
}
```

Resta.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>

#define N 10

void extraer(int matriz[N][N], char *str){
    char *token = strtok(str, " ");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = atoi(token);
            token = strtok(NULL, " ");
        }
    }
}

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Resta\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\tRealizo la resta de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    restarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("resta.txt", resultado);
    return 0;
}
```

Suma.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>

#define N 10

void extraer(int matriz[N][N], char *str){
    char *token = strtok(str, " ");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = atoi(token);
            token = strtok(NULL, " ");
        }
    }
}

void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Suma\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\tRealizo la suma de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    sumarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("suma.txt", resultado);
    return 0;
}
```

Transpuesta.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <string.h>

#define N 10

void extraer(int matriz[N][N], char *str){
    char *token = strtok(str, " ");
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            matriz[i][j] = atoi(token);
            token = strtok(NULL, " ");
        }
    }
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Transpuesta\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}
```

```
int main(int argc, char *argv[]){
    printf("\tRealizo la transposicion de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double t1[N][N];
    double t2[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    transponerMatriz(Matriz1, t1);
    transponerMatriz(Matriz2, t2);
    escribirMatriz("transpuesta_A.txt", t1);
    escribirMatriz("transpuesta_B.txt", t2);
    return 0;
}
```

Main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#include <string.h>
#define N 10
void imprimirMatriz(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}
void StringMatriz(int Matrizuno[N][N], char str[N*N*10]){
    str[0] = '\0';
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            sprintf(str + strlen(str), " %d", Matrizuno[i][j]);
        }
    }
}

int main() {
    srand(time(NULL));
    int A[N][N], B[N][N];
    double resultado[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    char Matriz1[N*N*10];
    char Matriz2[N*N*10];
    char *argv[4];
```

```

StringMatriz(A, Matriz1);
StringMatriz(B, Matriz2);
printf("Soy el proceso Padre. PID: %d\n", getpid());
int pidPadre=getpid();
pid_t pid;
for (int i = 0; i <= 5; i++) {
    pid = fork();
    sleep(1);
    if (pid == 0) {
        printf("Soy el proceso hijo %d con PID: %d, mi padre es el proceso raíz con PID: %d\n", i, getpid(), pidPadre);
        switch (i) {
            case 0:
                argv[0] = "suma";
                argv[1] = Matriz1;
                argv[2] = Matriz2;
                argv[3] = NULL;
                execv(argv[0], argv);
                break;
            case 1:
                argv[0] = "resta";
                argv[1] = Matriz1;
                argv[2] = Matriz2;
                argv[3] = NULL;
                execv(argv[0], argv);
                break;
            case 2:
                argv[0] = "multiplicacion";
                argv[1] = Matriz1;
                argv[2] = Matriz2;
                argv[3] = NULL;
                execv(argv[0], argv);
                break;
            case 3:
                argv[0] = "transpuesta";
                argv[1] = Matriz1;
                argv[2] = Matriz2;
                argv[3] = NULL;
                execv(argv[0], argv);
                break;

```

```

                case 4:
                    argv[0] = "inversa";
                    argv[1] = Matriz1;
                    argv[2] = Matriz2;
                    argv[3] = NULL;
                    execv(argv[0], argv);
                    break;
                case 5:
                    argv[0] = "archivo";
                    argv[1] = NULL;
                    execv(argv[0], argv);
                    break;
            }
            exit(0);
        }
    }
    else{
        wait(0);
    }
}
return 0;
}

```

```
said@said-VirtualBox:~/Descargas$ time ./main
```

```
Matriz A:
```

71	90	78	44	26	64	67	87	85	62
66	56	43	57	22	2	66	41	18	51
29	57	64	23	41	11	49	90	40	92
13	88	87	55	45	14	86	3	47	29
78	60	73	85	28	7	70	53	56	93
45	7	10	12	3	77	81	39	53	32
66	89	67	24	15	55	90	24	5	56
21	36	11	16	35	15	50	32	36	90
90	40	52	12	67	99	67	0	13	57
88	44	83	16	67	94	75	34	64	85

```
Matriz B:
```

65	15	60	75	79	46	49	61	31	98
99	84	7	34	1	60	93	9	51	10
14	97	43	14	85	6	67	92	12	28
95	45	95	4	72	41	37	15	26	88
6	93	50	79	76	42	61	22	74	67
74	98	10	23	8	47	37	54	23	81
5	1	42	89	16	27	29	67	36	75
33	92	27	89	9	82	24	48	74	13
57	46	64	58	68	43	87	2	11	96
93	69	94	81	52	22	0	27	12	97

```
Soy el proceso Padre. PID: 11273
```



```

Soy el proceso hijo 1 con PID: 11275, mi padre es el proceso raíz con PID: 11273
Realizo la resta de matrices
Soy el proceso hijo 2 con PID: 11285, mi padre es el proceso raíz con PID: 11273
Realizo la multiplicacion de matrices
Soy el proceso hijo 3 con PID: 11290, mi padre es el proceso raíz con PID: 11273
Realizo la transposicion de matrices
Soy el proceso hijo 4 con PID: 11294, mi padre es el proceso raíz con PID: 11273
Realizo la inversa de matrices
Soy el proceso hijo 5 con PID: 11298, mi padre es el proceso raíz con PID: 11273
Resultado Suma
136.00 105.00 138.00 119.00 105.00 110.00 116.00 148.00 116.00 160.00
165.00 140.00 50.00 91.00 23.00 62.00 159.00 50.00 69.00 61.00
43.00 154.00 107.00 37.00 126.00 17.00 116.00 182.00 52.00 120.00
108.00 133.00 182.00 59.00 117.00 55.00 123.00 18.00 73.00 117.00
84.00 153.00 123.00 164.00 104.00 49.00 131.00 75.00 130.00 160.00
119.00 105.00 20.00 35.00 11.00 124.00 118.00 93.00 76.00 113.00
71.00 90.00 109.00 113.00 31.00 82.00 119.00 91.00 41.00 131.00
43.00 113.00 63.00 100.00 25.00 117.00 39.00 98.00 106.00 49.00
147.00 86.00 116.00 70.00 135.00 142.00 154.00 2.00 24.00 153.00
181.00 113.00 177.00 97.00 119.00 116.00 75.00 61.00 76.00 182.00
Resultado Resta
6.00 75.00 18.00 -31.00 -53.00 18.00 18.00 26.00 54.00 -36.00
-33.00 -28.00 36.00 23.00 21.00 -58.00 -27.00 32.00 -33.00 41.00
15.00 -40.00 21.00 9.00 -44.00 5.00 -18.00 -2.00 28.00 64.00
-82.00 43.00 -8.00 51.00 -27.00 -27.00 49.00 -12.00 21.00 -59.00
72.00 -33.00 23.00 6.00 -48.00 -35.00 9.00 31.00 -18.00 26.00
-29.00 -91.00 0.00 -11.00 -5.00 30.00 44.00 -15.00 30.00 -49.00
61.00 88.00 25.00 -65.00 -1.00 28.00 61.00 -43.00 -31.00 -19.00
-23.00 -71.00 9.00 -78.00 7.00 -47.00 -9.00 2.00 -42.00 23.00
33.00 -6.00 -12.00 -46.00 -1.00 56.00 -20.00 -2.00 2.00 -39.00
-5.00 -25.00 -11.00 -65.00 15.00 72.00 75.00 7.00 52.00 -12.00

```

```

Resultado Multiplicacion
37506.00 43120.00 30795.00 36837.00 28844.00 29000.00 34083.00 27514.00 22796.00 41170.00
23583.00 22857.00 22561.00 24166.00 20018.00 17049.00 19312.00 17736.00 14794.00 27042.00
25720.00 33874.00 24932.00 30736.00 21746.00 20767.00 21830.00 20178.00 18427.00 28475.00
23211.00 28583.00 22179.00 22278.00 22109.00 16430.00 26603.00 18943.00 15200.00 27393.00
34733.00 34345.00 34481.00 33353.00 30972.00 23306.00 27579.00 24140.00 18971.00 40547.00
18303.00 18913.00 16094.00 22155.00 13035.00 15155.00 14898.00 16401.00 10946.00 27028.00
27214.00 29222.00 21056.00 26432.00 19214.00 18822.00 23459.00 23355.00 16131.00 29780.00
13861.00 19390.00 11902.00 14638.00 10897.00 11377.00 11952.00 10959.00 9112.00 15412.00
25783.00 30825.00 22400.00 27790.00 23238.00 18433.00 22882.00 23688.00 16240.00 36042.00
33166.00 41242.00 31121.00 37075.00 31325.00 24509.00 30681.00 29270.00 20444.00 45355.00
Resultado Transpuesta
71.00 66.00 29.00 13.00 78.00 45.00 66.00 10.00 90.00 88.00
90.00 56.00 57.00 88.00 60.00 7.00 89.00 21.00 40.00 44.00
78.00 43.00 64.00 87.00 73.00 10.00 67.00 36.00 52.00 83.00
44.00 57.00 23.00 55.00 85.00 12.00 24.00 11.00 12.00 16.00
26.00 22.00 41.00 45.00 28.00 3.00 15.00 16.00 67.00 67.00
64.00 2.00 11.00 14.00 7.00 77.00 55.00 35.00 99.00 94.00
67.00 66.00 49.00 86.00 70.00 81.00 90.00 15.00 67.00 75.00
87.00 41.00 90.00 3.00 53.00 39.00 24.00 50.00 0.00 34.00
85.00 18.00 40.00 47.00 56.00 53.00 5.00 32.00 13.00 64.00
62.00 51.00 92.00 29.00 93.00 32.00 56.00 36.00 57.00 85.00

```

Resultado Transpuesta

```
65.00 99.00 14.00 95.00 6.00 74.00 5.00 33.00 57.00 93.00
15.00 84.00 97.00 45.00 93.00 98.00 1.00 92.00 46.00 69.00
60.00 7.00 43.00 95.00 50.00 10.00 42.00 27.00 64.00 94.00
75.00 34.00 14.00 4.00 79.00 23.00 89.00 89.00 58.00 81.00
79.00 1.00 85.00 72.00 76.00 8.00 16.00 9.00 68.00 52.00
46.00 60.00 6.00 41.00 42.00 47.00 27.00 82.00 43.00 22.00
49.00 93.00 67.00 37.00 61.00 37.00 29.00 24.00 87.00 0.00
61.00 9.00 92.00 15.00 22.00 54.00 67.00 48.00 2.00 27.00
31.00 51.00 12.00 26.00 74.00 23.00 36.00 74.00 11.00 12.00
98.00 10.00 28.00 88.00 67.00 81.00 75.00 13.00 96.00 97.00
```

Resultado Inversa

```
-1.81 -0.87 0.83 -2.76 -0.56 1.56 -0.64 -2.16 -2.23 -0.13
1.29 -2.40 2.51 -1.24 -1.30 0.14 -0.43 -2.16 0.76 0.73
-1.51 1.62 -1.37 -1.48 -0.56 0.02 1.15 -2.62 -2.45 0.18
1.59 1.25 -0.53 2.48 -1.02 1.76 2.18 -2.48 -2.02 -0.24
-1.72 1.02 2.66 -2.16 0.41 -1.11 1.60 1.47 -1.55 1.05
-0.83 -2.40 1.57 -2.42 -2.23 0.14 0.14 -2.21 1.21 -0.69
-2.55 -2.56 2.12 2.41 0.36 1.23 0.98 -0.21 1.05 1.59
-2.78 0.06 -0.56 0.66 1.56 -0.37 -0.81 1.56 -1.92 1.63
-0.63 -2.65 -1.24 -1.90 -2.10 -2.64 0.34 -1.30 1.78 -1.58
2.23 -1.85 -2.50 -1.39 1.15 -0.86 -1.67 1.95 -0.49 2.08
```

Resultado Inversa

```
-5.78 -2.88 -0.34 0.53 3.22 1.92 0.11 -1.44 5.05 -1.34
-5.17 2.82 1.36 3.66 -5.38 5.04 -0.83 -2.63 3.54 4.83
2.97 1.63 4.30 -0.34 -0.49 5.65 1.45 -0.48 2.56 -1.46
-4.41 -0.90 5.83 3.99 2.11 1.71 -4.69 -0.06 -1.01 -4.26
-5.39 1.11 3.14 4.15 2.04 3.16 -2.57 -0.62 3.44 -5.09
5.46 -2.62 -1.80 -1.65 4.91 -0.38 -4.67 -2.18 -3.49 0.43
5.64 -2.26 2.26 0.39 -3.66 -2.37 1.58 4.71 -1.04 -4.65
-4.05 -4.31 4.28 -3.27 -2.24 1.33 3.41 3.13 -0.19 -2.65
4.96 -0.98 0.34 -0.46 4.43 -4.74 1.35 0.05 4.70 -2.53
-2.42 -5.75 1.29 -0.63 -0.06 -4.73 2.69 4.34 -2.59 -1.36
```

Tiempo:

```
real    0m11,714s
user    0m4,833s
sys     0m0,129s
```

La ejecución de un programa por copia exacta y otro por sustitución de código muestra diferencias en el tiempo de ejecución debido al modo en que se crean los procesos en cada caso. Cuando se usa copia exacta (`fork()`), el sistema operativo crea un nuevo proceso duplicando el espacio de memoria del proceso original, permitiendo que ambos procesos ejecuten el mismo código y compartan los recursos iniciales de manera eficiente. En contraste, la sustitución de código (`exec()`), reemplaza el código del proceso con el de un programa nuevo, lo que implica cargar y establecer nuevamente el entorno de ejecución, una operación más costosa en términos de tiempo. Por ello, en muchos casos, los procesos creados mediante copia exacta son más rápidos que aquellos creados por sustitución de código.

Sección Windows:

1. Inicie sesión en Windows.

2. Investigue en MSDN la llamada al sistema `CreateProcess()`, `WaitForSingleObject()` y `CloseHandle()`, reporte su funcionalidad, argumentos y retorno.

1. `CreateProcess()`

Función:

`CreateProcess` crea un nuevo proceso y su hilo principal. Este nuevo proceso puede ejecutar un programa especificado y heredar los identificadores de los objetos abiertos del proceso padre.

Sintaxis:

```
BOOL CreateProcess(  
    LPCSTR          lpApplicationName,  
    LPSTR           lpCommandLine,  
    LPSECURITY_ATTRIBUTES lpProcessAttributes,  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    BOOL            bInheritHandles,  
    DWORD           dwCreationFlags,  
    LPVOID          lpEnvironment,
```

```
LPCSTR          lpCurrentDirectory,  
  
LPSTARTUPINFO    lpStartupInfo,  
  
LPPROCESS_INFORMATION lpProcessInformation  
);
```

Argumentos principales:

- **lpApplicationName:** Ruta completa al archivo ejecutable.
- **lpCommandLine:** Línea de comandos para el nuevo proceso.
- **lpProcessAttributes:** Atributos de seguridad para el proceso.
- **lpThreadAttributes:** Atributos de seguridad para el hilo.
- **blInheritHandles:** Indica si el nuevo proceso hereda identificadores de su proceso padre.
- **dwCreationFlags:** Flags para controlar la creación del proceso (como prioridad y modo de ventana).
- **lpEnvironment:** Puntero al bloque de ambiente del proceso.
- **lpCurrentDirectory:** Directorio de trabajo inicial del proceso.
- **lpStartupInfo:** Información sobre la ventana y el comportamiento de inicio del proceso.
- **lpProcessInformation:** Estructura que recibe la información del nuevo proceso y su hilo principal.

Retorno:

Retorna un valor booleano (TRUE si se crea el proceso con éxito, FALSE si falla). Si falla, se puede obtener más información del error con GetLastError().

2. WaitForSingleObject()

Función:

WaitForSingleObject espera a que un objeto especificado (como un proceso o un hilo) cambie a un estado señalado o se agote el tiempo especificado.

Sintaxis:

```
DWORD WaitForSingleObject(  
  
HANDLE hHandle,  
  
DWORD dwMilliseconds  
);
```

Argumentos:

- **hHandle:** Identificador del objeto en espera (por ejemplo, el proceso o hilo creado anteriormente).
- **dwMilliseconds:** Tiempo de espera en milisegundos (usar INFINITE para esperar indefinidamente).

Retorno:

Retorna un código DWORD que indica el estado del objeto:

- **WAIT_OBJECT_0:** El objeto cambió a un estado señalado.
 - **WAIT_TIMEOUT:** Se agotó el tiempo especificado sin que el objeto cambiara de estado.
 - Otros códigos indican errores.
-

3. CloseHandle()**Función:**

CloseHandle cierra un identificador de objeto del sistema. Este identificador ya no puede ser usado, y los recursos asociados se liberan.

Sintaxis:

```
BOOL CloseHandle(  
  
    HANDLE hObject  
  
);
```

Argumentos:

- **hObject:** Identificador del objeto a cerrar (por ejemplo, un identificador de proceso o hilo).

Retorno:

Retorna TRUE si el identificador fue cerrado correctamente, o FALSE en caso de error (más detalles de error pueden obtenerse con GetLastError()).

3. Capture y compile el programa de creación de un nuevo proceso que a continuación se muestra.

```
#include <windows.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    STARTUPINFO si; /* Estructura de información inicial para Windows */
    PROCESS_INFORMATION pi; /* Estructura de información del adm. de procesos */
    int i;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    if(argc!=2)
    {
        printf("Usar: %s Nombre_programa_hijo\n", argv[0]);
        return;
    }
    // Creación proceso hijo
    if(!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi))
    {
        printf( "Fallo al invocar CreateProcess (%d)\n", GetLastError() );
        return;
    }
    // Proceso padre
    printf("Soy el padre\n");
    WaitForSingleObject(pi.hProcess, INFINITE);
    // Terminación controlada del proceso e hilo asociado de ejecución
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

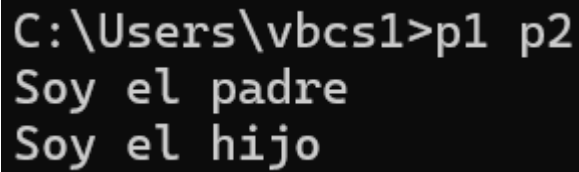
```
C:\Users\vbcs1>gcc p1w.c -o p1
```

4. Capture y compile el programa que contendrá al proceso hijo que a continuación se muestra.

```
#include <windows.h>
#include <stdio.h>
int main(void)
{
    printf("Soy el hijo\n");
    exit(0);
}
```

```
C:\Users\vbcs1>gcc p2w.c -o p2
```

5. Ejecute el primer código pasando como argumento a través de la línea de comando, el nombre del archivo ejecutable del segundo código capturado. Observe el funcionamiento del programa, reporte sus observaciones y experimente con el código.



```
C:\Users\vbcs1>p1 p2
Soy el padre
Soy el hijo
```

Observaciones:

- El código del proceso padre crea un proceso hijo utilizando la función `CreateProcess()`.
- Utiliza las estructuras `STARTUPINFO` y `PROCESS_INFORMATION` para iniciar y administrar el proceso hijo. `ZeroMemory` se usa para inicializar estas estructuras, y el campo `cb` de `STARTUPINFO` se configura con su tamaño, como requiere `CreateProcess()` para funcionar correctamente.
- Verifica si se proporciona el nombre del programa hijo como argumento de la línea de comandos. Si no se pasa el nombre del programa, imprime un mensaje de uso y termina la ejecución del programa padre.
- Imprime un mensaje indicando que es el proceso padre. Esta impresión permite al usuario diferenciar entre la ejecución del proceso padre y el hijo en la consola.
- Espera a que el proceso hijo finalice usando `WaitForSingleObject()`. Este código utiliza `INFINITE` como tiempo de espera, lo cual hace que el proceso padre espere indefinidamente hasta que el hijo termine. Esto asegura que el padre no continúe hasta que el hijo haya terminado su ejecución.
- Cierra los identificadores de proceso e hilo asociados al proceso hijo usando `CloseHandle()`. Esta liberación de recursos es esencial para evitar fugas de memoria y asegurar una gestión adecuada de los identificadores del sistema.
- El código del proceso hijo imprime un mensaje indicando que es el proceso hijo. Esto confirma que el proceso hijo se ejecutó correctamente y ayuda en la depuración para verificar la creación del proceso.
- Sale del proceso hijo usando `exit(0)`, lo que indica que termina con éxito. Además, permite al proceso padre continuar con la limpieza final y la liberación de recursos asociados.

6. Compare y reporte tanto las diferencias como similitudes que encuentra con respecto a la creación de procesos por sustitución de código en Linux.

Similitudes

1.Objetivo de Crear Nuevos Procesos:

Ambos enfoques, en Windows y Linux, buscan crear un nuevo proceso que ejecute un programa específico. En ambos sistemas, el proceso hijo puede ejecutar una tarea diferente del proceso padre.

2.Separación de Procesos:

En ambos sistemas, el proceso padre puede ejecutar otras tareas o esperar a que el proceso hijo termine, utilizando funciones como WaitForSingleObject en Windows y waitpid en Linux para sincronización.

3.Intercambio de Información:

Tanto en Windows como en Linux, los procesos hijo pueden compartir información con el padre (por medio de variables de entorno, archivos temporales, o tuberías) aunque, por defecto, cada proceso tiene su propio espacio de memoria.

4.Liberación de Recursos:

Ambos sistemas operativos requieren que los recursos utilizados por los procesos (como identificadores de procesos o punteros) se liberen para evitar fugas de memoria. Windows usa CloseHandle(), mientras que en Linux, el sistema se asegura de que los identificadores de procesos (PIDs) y otros recursos se liberen al finalizar.

Diferencias

1.Creación del Proceso:

En Windows, CreateProcess() realiza tanto la creación como la ejecución del programa en un solo paso, especificando tanto el entorno como la prioridad del nuevo proceso.

En Linux, el modelo se divide en dos pasos: primero, fork() crea una copia del proceso padre; luego, el proceso hijo ejecuta exec() para reemplazar su código con el del programa especificado. Este enfoque en dos pasos da flexibilidad para que el proceso hijo modifique su entorno antes de ejecutar el nuevo programa.

2.Función de Creación y Ejecución:

Windows: CreateProcess() gestiona todos los aspectos de la creación del proceso, incluyendo el archivo ejecutable, los argumentos, el entorno, y la configuración de inicio. Además, Windows utiliza estructuras como STARTUPINFO y PROCESS_INFORMATION para proporcionar una configuración detallada.

Linux: fork() y exec() permiten una separación. fork() crea un clon del proceso padre, y luego exec() sustituye el espacio de código del proceso hijo. Esto permite al hijo hacer modificaciones (como cerrar o redirigir archivos) antes de ejecutar el nuevo programa.

3.Simultaneidad en la Creación de Procesos:

Linux: fork() crea un proceso completamente idéntico al padre, compartiendo la mayoría de recursos de forma temporal, y exec() luego reemplaza el espacio de código del hijo sin afectar al padre.

Windows: CreateProcess() inicia el nuevo proceso directamente con su propio espacio de código y contexto desde el inicio, evitando la duplicación de recursos, lo cual puede ser más eficiente en términos de memoria.

4.Información de Estado y Control:

Windows: La estructura PROCESS_INFORMATION proporciona identificadores de proceso e hilo que permiten al padre manipular o monitorizar directamente el proceso hijo.

Linux: Tras fork(), el padre recibe un PID del hijo que permite controlarlo, y funciones como waitpid() pueden monitorear el estado del hijo. Sin embargo, exec() no proporciona ningún control adicional sobre el proceso hijo una vez que ha reemplazado su código.

5.Comunicación entre Procesos:

Windows: Aunque no es parte de CreateProcess(), Windows permite opciones para heredar identificadores y variables de entorno o utilizar pipes explícitamente configuradas.

Linux: fork() hereda automáticamente todos los recursos abiertos del padre, y las tuberías (pipes) pueden configurarse antes de la bifurcación, permitiendo una comunicación directa entre procesos.

6.Control de Prioridad:

Windows: CreateProcess() permite establecer la prioridad del proceso hijo desde su creación.

Linux: La prioridad puede modificarse después de crear el proceso hijo usando funciones como nice() o setpriority().

7. Programe una aplicación que cree un proceso hijo a partir de un proceso padre, el hijo creado a su vez creará 5 procesos hijos más. A su vez cada uno de los cinco procesos creará 3 procesos más. Cada uno de los procesos creados imprimirá en pantalla su identificador. CONSEJO: INVESTIGUE LA FUNCIÓN GetCurrentProcessId() DEL API WIN32.

Padre.c

```
#include <windows.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    printf("Proceso padre (nivel 0), PID: %lu\n", GetCurrentProcessId());
    if (!CreateProcess(NULL, argv[1], NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
        printf("Fallo al invocar CreateProcess nivel 1 (%d)\n", GetLastError());
        return 1;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

Hijos.c

```
#include <windows.h>
#include <stdio.h>
int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    printf("\tProceso hijo con PID: %d\n", GetCurrentProcessId());
    for (int i = 0; i < 5; i++) {
        ZeroMemory(&si, sizeof(si));
        si.cb = sizeof(si);
        ZeroMemory(&pi, sizeof(pi));
        if (!CreateProcess(NULL, "nietos", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
            printf("Fallo al invocar CreateProcess en nivel (%d)\n", GetLastError());
        } else {
            WaitForSingleObject(pi.hProcess, INFINITE);
            CloseHandle(pi.hProcess);
            CloseHandle(pi.hThread);
        }
    }
    return 0;
}
```

Nietos.c

```
#include <windows.h>
#include <stdio.h>
int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    printf("\t\tProceso nieto con PID: %d\n", GetCurrentProcessId());
    for (int i = 0; i < 3; i++) {
        ZeroMemory(&si, sizeof(si));
        si.cb = sizeof(si);
        ZeroMemory(&pi, sizeof(pi));
        if (!CreateProcess(NULL, "bisnietos", NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
            printf("Fallo al invocar CreateProcess en nivel (%d)\n", GetLastError());
        } else {
            WaitForSingleObject(pi.hProcess, INFINITE);
            CloseHandle(pi.hProcess);
            CloseHandle(pi.hThread);
        }
    }
    return 0;
}
```

Bisnietos.c

```
#include <windows.h>
#include <stdio.h>
int main(){
    printf("\t\t\tProceso bisnieto, Mi PID es: %d\n", GetCurrentProcessId());
    return 0;
}
```

```
C:\Users\vbcs1>padre hijo
Proceso padre (nivel 0), PID: 9636
    Proceso hijo con PID: 13212
        Proceso nieto con PID: 4132
            Proceso bisnieto, Mi PID es: 4236
            Proceso bisnieto, Mi PID es: 15696
            Proceso bisnieto, Mi PID es: 18328
        Proceso nieto con PID: 8212
            Proceso bisnieto, Mi PID es: 20172
            Proceso bisnieto, Mi PID es: 13248
            Proceso bisnieto, Mi PID es: 24140
        Proceso nieto con PID: 25384
            Proceso bisnieto, Mi PID es: 11916
            Proceso bisnieto, Mi PID es: 14464
            Proceso bisnieto, Mi PID es: 25312
        Proceso nieto con PID: 13628
            Proceso bisnieto, Mi PID es: 6508
            Proceso bisnieto, Mi PID es: 15324
            Proceso bisnieto, Mi PID es: 24296
        Proceso nieto con PID: 17136
            Proceso bisnieto, Mi PID es: 21788
            Proceso bisnieto, Mi PID es: 22156
            Proceso bisnieto, Mi PID es: 6484
```

8. Programe la aplicación desarrollada en el punto 5 de la sección de Linux utilizando esta vez la creación de procesos en Windows. Obtenga el tiempo de ejecución de la aplicación programada, compare este tiempo contra el tiempo obtenido del programa de creación de procesos por sustitución de código de Linux, y dé sus observaciones.

Main.c

```
#include <windows.h>
#include <stdio.h>
#include <time.h>
#define N 10
void imprimirMatriz(int A[N][N]){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            printf("%d\t",A[i][j]);
        }
        printf("\n");
    }
}
void StringMatriz(int Matrizuno[N][N], char str[N*N*10]){
    str[0] = '\0';
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            sprintf(str + strlen(str), " %d", Matrizuno[i][j]);
        }
    }
}
int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    srand(time(NULL));
    int A[N][N], B[N][N];
    double resultado[N][N];
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 101;
            B[i][j] = rand() % 101;
        }
    }
    printf("\nMatriz A:\n");
    imprimirMatriz(A);
    printf("\nMatriz B:\n");
    imprimirMatriz(B);
    char Matriz1[N*N*10];
    char Matriz2[N*N*10];
    char comando[N*N*10];
    StringMatriz(A, Matriz1);
    StringMatriz(B, Matriz2);
```

```

for (int i = 0; i <= 5; i++) {
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    switch (i) {
        case 0:
            sprintf(comando, "%s \"%s\" \"%s\"", "suma", Matriz1, Matriz2);
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
        case 1:
            sprintf(comando, "%s \"%s\" \"%s\"", "resta", Matriz1, Matriz2);
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
        case 2:
            sprintf(comando, "%s \"%s\" \"%s\"", "multiplicacion", Matriz1, Matriz2);
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
        case 3:
            sprintf(comando, "%s \"%s\" \"%s\"", "transpuesta", Matriz1, Matriz2);
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
        case 4:
            sprintf(comando, "%s \"%s\" \"%s\"", "inversa", Matriz1, Matriz2);
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
    }
}

```

```

        case 5:
            sprintf(comando, "archivo");
            if (!CreateProcess(NULL, comando, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
                printf("Fallo al invocar CreateProcess (%d)\n", GetLastError());
                return 1;
            }
            break;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
return 0;
}

```

Archivo.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
void mostrar(char *archivo){
    int i = 0, j = 0;
    char num;
    FILE *file = fopen(archivo, "r");
    if (file == NULL){
        printf("Error al abrir el archivo %s\n", archivo);
        return;
    }
    while ((num = fgetc(file)) != EOF && j < 10) {
        printf("%c", num);
        if (num == '\n') {
            j++;
            printf("\n");
        }
        i++;
    }
    fclose(file);
}
int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 6 con PID: %d \n", GetCurrentProcessId());
    mostrar("suma.txt");
    mostrar("resta.txt");
    mostrar("multiplicacion.txt");
    mostrar("transpuesta_A.txt");
    mostrar("transpuesta_B.txt");
    mostrar("inversa_A.txt");
    mostrar("inversa_B.txt");
    return 0;
}
```

Inversa.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define N 10
void extraer(int matriz[N][N], char str[]){
    int fila = 0, columna = -1;
    int i = 0;
    int num = 0;
    char num_str[10] = {'\0'};
    while (str[i] != '\0'){
        if (str[i] == ' '){
            matriz[fila][columna++] = atoi(num_str);
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        else if (str[i] >= '0' && str[i] <= '9'){
            num_str[num++] = str[i];
        }
        else if (str[i] == '\n'){
            matriz[fila][columna++] = atoi(num_str);
            fila++;
            columna = 0;
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        i++;
    }
    matriz[fila][columna] = atoi(num_str);
}
```

```

void Cofactor(int matriz[N][N], int temp[N][N], int p, int q, int n) {
    int i = 0, j = 0;
    for (int fila = 0; fila < n; fila++) {
        for (int col = 0; col < n; col++) {
            if (fila != p && col != q) {
                temp[i][j++] = matriz[fila][col];
                if (j == n - 1) {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int DeterminanteMatriz(int matriz[N][N], int n) {
    int determinante = 0;
    if (n == 1)
        return matriz[0][0];

    int temp[N][N];
    int signo = 1;
    for (int f = 0; f < n; f++) {
        Cofactor(matriz, temp, 0, f, n);
        determinante += signo * matriz[0][f] * DeterminanteMatriz(temp, n - 1);
        signo = -signo;
    }
    return determinante;
}

```

```

void AdjuntaMatriz(int matriz[N][N], double adjunta[N][N], int n) {
    if (n == 1) {
        adjunta[0][0] = 1;
        return;
    }

    int temp[N][N];
    int signo = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            Cofactor(matriz, temp, i, j, n);
            signo = ((i + j) % 2 == 0) ? 1 : -1;
            adjunta[j][i] = (signo) * (DeterminanteMatriz(temp, n - 1));
        }
    }
}

void InversaMatriz(int matriz[N][N], double inversa[N][N]) {
    double determinante = (double)DeterminanteMatriz(matriz, N);
    if (determinante == 0) {
        printf("La matriz es singular, no se puede calcular la inversa.\n");
        return;
    }
    double adjunta[N][N];
    AdjuntaMatriz(matriz, adjunta, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            inversa[i][j] = adjunta[i][j] / determinante;
}

```



```

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Inversa\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 5 con PID: %d \n", GetCurrentProcessId());
    printf("\tRealizo la inversa de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double Inversa1[N][N];
    double Inversa2[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    InversaMatriz(Matriz1, Inversa1);
    InversaMatriz(Matriz2, Inversa2);
    escribirMatriz("inversa_A.txt", Inversa1);
    escribirMatriz("inversa_B.txt", Inversa2);
    return 0;
}

```

Multiplicación.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define N 10
void extraer(int matriz[N][N], char str[]){
    int fila = 0, columna = -1;
    int i = 0;
    int num = 0;
    char num_str[10] = {'\0'};
    while (str[i] != '\0'){
        if (str[i] == ' '){
            matriz[fila][columna++] = atoi(num_str);
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        else if (str[i] >= '0' && str[i] <= '9'){
            num_str[num++] = str[i];
        }
        else if (str[i] == '\n'){
            matriz[fila][columna++] = atoi(num_str);
            fila++;
            columna = 0;
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        i++;
    }
    matriz[fila][columna] = atoi(num_str);
}
```

```

void multiplicarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = 0;
            for (int k = 0; k < N; k++) {
                resultado[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Multiplicacion\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 3 con PID: %d \n", GetCurrentProcessId());
    printf("\tRealizo la multiplicacion de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    multiplicarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("multiplicacion.txt", resultado);
    return 0;
}

```

Resta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define N 10
void extraer(int matriz[N][N], char str[]){
    int fila = 0, columna = -1;
    int i = 0;
    int num = 0;
    char num_str[10] = {'\0'};
    while (str[i] != '\0'){
        if (str[i] == ' '){
            matriz[fila][columna++] = atoi(num_str);
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        else if (str[i] >= '0' && str[i] <= '9'){
            num_str[num++] = str[i];
        }
        else if (str[i] == '\n'){
            matriz[fila][columna++] = atoi(num_str);
            fila++;
            columna = 0;
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        i++;
    }
    matriz[fila][columna] = atoi(num_str);
}

void restarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] - B[i][j];
        }
    }
}
```

```

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Resta\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 2 con PID: %d \n", GetCurrentProcessId());
    printf("\tRealizo la resta de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    restarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("resta.txt", resultado);
    return 0;
}

```

Suma.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define N 10
void extraer(int matriz[N][N], char str[]){
    int fila = 0, columna = -1;
    int i = 0;
    int num = 0;
    char num_str[10] = {'\0'};
    while (str[i] != '\0'){
        if (str[i] == ' '){
            matriz[fila][columna++] = atoi(num_str);
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        else if (str[i] >= '0' && str[i] <= '9'){
            num_str[num++] = str[i];
        }
        else if (str[i] == '\n'){
            matriz[fila][columna++] = atoi(num_str);
            fila++;
            columna = 0;
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        i++;
    }
    matriz[fila][columna] = atoi(num_str);
}

void sumarMatrices(int A[N][N], int B[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[i][j] = A[i][j] + B[i][j];
        }
    }
}
```

```

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Suma\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 1 con PID: %d \n", GetCurrentProcessId());
    printf("\tRealizo la suma de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double resultado[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    sumarMatrices(Matriz1, Matriz2, resultado);
    escribirMatriz("suma.txt", resultado);
    return 0;
}

```

Transpuesta.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>

#define N 10
void extraer(int matriz[N][N], char str[]){
    int fila = 0, columna = -1;
    int i = 0;
    int num = 0;
    char num_str[10] = {'\0'};
    while (str[i] != '\0'){
        if (str[i] == ' '){
            matriz[fila][columna++] = atoi(num_str);
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        else if (str[i] >= '0' && str[i] <= '9'){
            num_str[num++] = str[i];
        }
        else if (str[i] == '\n'){
            matriz[fila][columna++] = atoi(num_str);
            fila++;
            columna = 0;
            num = 0;
            memset(num_str, '\0', sizeof(num_str));
        }
        i++;
    }
    matriz[fila][columna] = atoi(num_str);
}

void transponerMatriz(int matriz[N][N], double resultado[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            resultado[j][i] = matriz[i][j];
        }
    }
}
```



```

void escribirMatriz(char *nombre_archivo, double matriz[N][N]) {
    FILE *archivo = fopen(nombre_archivo, "w");
    if (archivo == NULL) {
        perror("Error al abrir el archivo");
        exit(1);
    }
    fprintf(archivo, "\tResultado Transpuesta\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(archivo, "%.2f ", matriz[i][j]);
        }
        fprintf(archivo, "\n");
    }
    fclose(archivo);
}

int main(int argc, char *argv[]){
    printf("\nSoy el proceso hijo 4 con PID: %d \n", GetCurrentProcessId());
    printf("\tRealizo la transposicion de matrices\n");
    int Matriz1[N][N];
    int Matriz2[N][N];
    double t1[N][N];
    double t2[N][N];
    extraer(Matriz1, argv[1]);
    extraer(Matriz2, argv[2]);
    transponerMatriz(Matriz1, t1);
    transponerMatriz(Matriz2, t2);
    escribirMatriz("transpuesta_A.txt", t1);
    escribirMatriz("transpuesta_B.txt", t2);
    return 0;
}

```

```
C:\Users\vbcsl>gcc archivo.c -o archivo
```

```
C:\Users\vbcsl>main
```

```
Matriz A:
```

83	48	58	92	77	2	57	63	62	27
60	68	5	28	88	65	72	26	70	80
66	54	4	68	60	43	70	95	20	71
43	46	96	0	7	51	72	98	50	34
99	36	31	3	44	10	73	39	75	27
76	13	12	57	56	43	49	27	34	31
83	68	10	65	100	30	83	6	20	98
81	22	7	2	37	33	95	36	77	13
3	30	5	88	41	93	100	46	29	43
2	11	54	43	91	56	62	0	34	44

```
Matriz B:
```

74	86	61	46	26	85	67	91	65	91
78	34	1	0	35	90	21	51	31	85
57	57	80	54	63	92	81	95	50	78
39	55	82	43	3	51	5	89	66	13
60	7	63	78	14	6	1	98	20	4
51	88	30	57	72	36	86	49	45	39
53	55	97	14	90	15	85	60	2	8
25	63	30	97	39	66	97	63	51	7
14	69	6	38	97	58	91	99	40	56
26	53	76	9	8	55	65	13	88	9

```
Soy el proceso hijo 1 con PID: 6024
```

```
Realizo la suma de matrices
```

```
Soy el proceso hijo 2 con PID: 12108
```

```
Realizo la resta de matrices
```

```
Soy el proceso hijo 3 con PID: 12828
```

```
Realizo la multiplicacion de matrices
```

```
Soy el proceso hijo 4 con PID: 22556
```

```
Realizo la transposicion de matrices
```

```
Soy el proceso hijo 5 con PID: 18884
```

```
Realizo la inversa de matrices
```

```
Soy el proceso hijo 6 con PID: 23004
```

Resultado Suma

157.00 134.00 119.00 138.00 103.00 87.00 124.00 154.00 127.00 118.00
138.00 102.00 6.00 28.00 123.00 155.00 93.00 77.00 101.00 165.00
123.00 111.00 84.00 122.00 123.00 135.00 151.00 190.00 70.00 149.00
82.00 101.00 178.00 43.00 10.00 102.00 77.00 187.00 116.00 47.00
159.00 43.00 94.00 81.00 58.00 16.00 74.00 137.00 95.00 31.00
127.00 101.00 42.00 114.00 128.00 79.00 135.00 76.00 79.00 70.00
136.00 123.00 107.00 79.00 190.00 45.00 168.00 66.00 22.00 106.00
106.00 85.00 37.00 99.00 76.00 99.00 192.00 99.00 128.00 20.00
17.00 99.00 11.00 126.00 138.00 151.00 191.00 145.00 69.00 99.00

Resultado Resta

9.00 -38.00 -3.00 46.00 51.00 -83.00 -10.00 -28.00 -3.00 -64.00
-18.00 34.00 4.00 28.00 53.00 -25.00 51.00 -25.00 39.00 -5.00
9.00 -3.00 -76.00 14.00 -3.00 -49.00 -11.00 0.00 -30.00 -7.00
4.00 -9.00 14.00 -43.00 4.00 0.00 67.00 9.00 -16.00 21.00
39.00 29.00 -32.00 -75.00 30.00 4.00 72.00 -59.00 55.00 23.00
25.00 -75.00 -18.00 0.00 -16.00 7.00 -37.00 -22.00 -11.00 -8.00
30.00 13.00 -87.00 51.00 10.00 15.00 -2.00 -54.00 18.00 90.00
56.00 -41.00 -23.00 -95.00 -2.00 -33.00 -2.00 -27.00 26.00 6.00
-11.00 -39.00 -1.00 50.00 -56.00 35.00 9.00 -53.00 -11.00 -13.00

Resultado Multiplicacion

27668.00 30664.00 32049.00 26534.00 22807.00 32031.00 30329.00 45221.00 25668.00 22351.00
27242.00 30301.00 28182.00 21713.00 25175.00 27232.00 31883.00 37632.00 24101.00 20279.00
25980.00 30662.00 30202.00 24901.00 20511.00 28599.00 31578.00 36267.00 25820.00 16693.00
23113.00 30657.00 25128.00 23335.00 27970.00 30823.00 38402.00 34450.00 21590.00 21696.00
21764.00 25936.00 22626.00 18257.00 22714.00 24788.00 29387.00 33472.00 18140.00 20392.00
19652.00 23358.00 23224.00 18290.00 16247.00 19719.00 22269.00 29797.00 18323.00 14363.00
29458.00 28452.00 34260.00 20049.00 19351.00 27281.00 26631.00 37638.00 25539.00 19236.00
19441.00 24881.00 20753.00 16822.00 23744.00 20003.00 28760.00 30339.00 14904.00 16689.00
21456.00 27552.00 27724.00 20042.00 22928.00 20080.00 28111.00 31013.00 19678.00 11281.00

Resultado Transpuesta

83.00	60.00	66.00	43.00	99.00	76.00	83.00	81.00	3.00	2.00
48.00	68.00	54.00	46.00	36.00	13.00	68.00	22.00	30.00	11.00
58.00	5.00	4.00	96.00	31.00	12.00	10.00	7.00	5.00	54.00
92.00	28.00	68.00	0.00	3.00	57.00	65.00	2.00	88.00	43.00
77.00	88.00	60.00	7.00	44.00	56.00	100.00	37.00	41.00	91.00
2.00	65.00	43.00	51.00	10.00	43.00	30.00	33.00	93.00	56.00
57.00	72.00	70.00	72.00	73.00	49.00	83.00	95.00	100.00	62.00
63.00	26.00	95.00	98.00	39.00	27.00	6.00	36.00	46.00	0.00
62.00	70.00	20.00	50.00	75.00	34.00	20.00	77.00	29.00	34.00

Resultado Transpuesta

74.00	78.00	57.00	39.00	60.00	51.00	53.00	25.00	14.00	26.00
86.00	34.00	57.00	55.00	7.00	88.00	55.00	63.00	69.00	53.00
61.00	1.00	80.00	82.00	63.00	30.00	97.00	30.00	6.00	76.00
46.00	0.00	54.00	43.00	78.00	57.00	14.00	97.00	38.00	9.00
26.00	35.00	63.00	3.00	14.00	72.00	90.00	39.00	97.00	8.00
85.00	90.00	92.00	51.00	6.00	36.00	15.00	66.00	58.00	55.00
67.00	21.00	81.00	5.00	1.00	86.00	85.00	97.00	91.00	65.00
91.00	51.00	95.00	89.00	98.00	49.00	60.00	63.00	99.00	13.00
65.00	31.00	50.00	66.00	20.00	45.00	2.00	51.00	40.00	88.00

Resultado Inversa

-0.25 -1.03 0.13 -0.01 0.38 -1.05 0.16 -1.12 0.17 -0.75
-0.90 -0.56 -0.49 -0.83 -0.75 -0.33 -0.93 -0.46 0.70 0.32
-1.10 0.09 -0.78 -0.44 -0.62 0.95 0.85 0.61 -0.71 -0.53
0.16 -0.72 -0.26 0.44 -1.06 0.83 0.04 0.05 -0.89 0.67
0.61 -0.77 0.86 0.61 0.41 0.84 -0.24 1.06 1.14 0.65
-0.23 -0.50 -0.31 0.15 0.66 -0.18 -0.30 0.37 0.39 -1.14
1.09 -0.07 -0.82 0.88 -0.63 0.68 0.46 0.95 -0.28 0.24
-0.11 1.02 0.46 0.78 -0.20 1.11 0.79 0.42 0.90 -0.18
-0.60 -0.67 -0.12 0.11 -0.93 -0.91 -0.70 -1.05 -0.16 -0.01

Resultado Inversa

0.85 -6.42 -5.21 -1.47 5.84 1.03 5.81 5.27 6.05 1.18
-0.52 -4.82 4.96 3.50 -3.35 6.66 -2.42 -3.65 2.39 -2.58
-3.72 4.84 3.62 1.31 3.18 1.96 -5.16 4.06 4.50 2.90
-3.25 -5.19 6.20 -1.17 -0.92 -0.55 6.83 -6.38 -1.71 4.06
-3.02 -2.71 4.76 5.43 -0.46 1.17 5.98 -5.13 3.23 -1.00
6.50 -4.48 3.58 -0.53 -1.47 0.45 5.87 -0.84 -4.79 -2.39
-2.00 -6.41 5.91 3.70 -1.06 -4.15 -1.74 -0.45 -5.16 0.59
2.40 -5.61 0.57 -1.24 -3.21 4.86 4.63 -5.09 5.22 2.17
0.23 6.16 -0.76 -6.40 -2.74 6.64 6.61 1.87 -4.99 -1.33

Tiempo en Linux vs Windows

```
real    0m11,714s
user    0m4,833s
sys     0m0,129s
```

```
Seconds      : 4
Milliseconds : 201
Ticks        : 42014814
```

Observaciones:

- Tiempo de ejecución en Linux (creación de procesos por sustitución de código): 4.833 segundos.
- Tiempo de ejecución en Windows (creación de procesos): 4.201 segundos.

En Windows, la aplicación se ejecuta directamente en el hardware. En Linux, se ejecuta en una máquina virtual, lo que introduce una capa adicional de virtualización. Windows y Linux manejan los procesos de manera distinta; en Linux, `fork()` y `exec()` duplican el proceso, lo cual puede ser más lento que el `CreateProcess()` de Windows, especialmente si hay muchos procesos involucrados. La máquina virtual en Linux tiene acceso indirecto al hardware y recursos limitados, lo que ralentiza la creación de procesos en comparación con la ejecución directa en Windows.

Debido a la combinación de virtualización y el método de creación de procesos, la misma aplicación tarda 8 segundos en Linux y solo 4 segundos en Windows. Esta diferencia es normal porque la virtualización y el manejo de procesos en Linux introducen retrasos adicionales en comparación con la ejecución directa en Windows.

Conclusión

La práctica realizada sobre la administración de procesos en Linux y Windows, en la que se desarrollaron programas para realizar operaciones sobre matrices (suma, resta, multiplicación, transposición e inversa) y se implementaron estructuras de árbol para la gestión de procesos, ha proporcionado una valiosa experiencia en el análisis comparativo de ambos sistemas operativos en términos de rendimiento y manejo de procesos. A continuación se detallan los hallazgos y conclusiones más relevantes de esta experiencia.

Comparación de Rendimiento

Durante la ejecución de los programas en ambos sistemas operativos, se observó una diferencia notable en los tiempos de creación de procesos. En Windows, la creación de procesos tardó aproximadamente 4.2 segundos, mientras que en Linux, el mismo proceso en una máquina virtual tardó cerca de 4.8 segundos. Este resultado se puede atribuir a varios factores:

1. **Manejo de Procesos:** Windows utiliza el método `CreateProcess()`, que está optimizado para la creación rápida de procesos. Por otro lado, Linux usa `fork()` y `exec()`, que pueden ser más lentos, especialmente en un entorno de virtualización donde la sobrecarga de recursos es mayor.
2. **Virtualización:** La ejecución de Linux en una máquina virtual introdujo una capa adicional que complicó el acceso directo al hardware. Esto resultó en una latencia adicional en la creación de procesos y el acceso a recursos, lo que no se experimentó en la instalación nativa de Windows.
3. **Optimización de Recursos:** En un entorno virtualizado, los recursos como CPU y memoria están limitados, lo que puede afectar el rendimiento general de las aplicaciones. La configuración de la máquina virtual influyó en la capacidad del sistema para gestionar múltiples procesos de manera eficiente.

Implementación de Operaciones con Matrices

La implementación de las operaciones sobre matrices proporcionó una plataforma ideal para medir el rendimiento y la eficiencia de los procesos en ambos sistemas. Las operaciones de suma, resta, multiplicación y transposición se ejecutaron correctamente en ambos sistemas, pero la rapidez y la eficiencia en la gestión de memoria variaron. Windows mostró un rendimiento superior, posiblemente debido a su manejo más directo y optimizado de los recursos.

Estructura de Árboles de Procesos

La creación de árboles de procesos permitió observar cómo ambos sistemas operativos gestionan y representan la jerarquía de los procesos. En Linux, la

utilización de herramientas como pstree y top facilitó la visualización de los procesos y su relación jerárquica, mientras que en Windows, el Administrador de Tareas ofreció una interfaz gráfica que permitió una fácil identificación de los procesos y sus árboles de dependencia.

Consideraciones sobre la Gestión de Archivos

Ambos sistemas operativos realizaron operaciones de entrada/salida (E/S) para guardar resultados en archivos y mostrarlos en la línea de comandos. Sin embargo, se observó que la velocidad de lectura y escritura podía variar según la configuración del sistema de archivos y el entorno. La diferencia en el sistema de archivos (NTFS en Windows frente a ext4 o otros en Linux) también puede influir en el rendimiento de las operaciones de E/S.

La práctica evidenció que, aunque el código fuente era idéntico en ambos sistemas operativos, las diferencias en el manejo de procesos, la virtualización y la optimización de recursos generaron variaciones significativas en el rendimiento. Windows demostró ser más eficiente en la creación de procesos y la ejecución de operaciones intensivas en recursos. Por otro lado, Linux, aunque un poco más lento en un entorno virtualizado, mostró su robustez y flexibilidad, especialmente en la gestión de procesos y estructuras de árbol.

Estos hallazgos subrayan la importancia de considerar el entorno y la plataforma al desarrollar aplicaciones y realizar análisis de rendimiento. La elección entre Windows y Linux debe basarse en los requisitos específicos de la aplicación, el entorno de ejecución y las necesidades de rendimiento, así como en las características del sistema operativo que mejor se adapten al caso de uso previsto. La experiencia adquirida en esta práctica es valiosa para futuros desarrollos y para la toma de decisiones informadas en la administración de sistemas y recursos.